

1. INTRODUCTION

With explosive growth of mobile devices including smart phones, PDAs, and tablet computers and the applications installed in them, the mobile-Internet will maintain the development growth trend as 4G communication network is extensively promoted to our lives. What users of the mobile devices and applications need is that mobile-Internet can provide them with the service which is user-friendly, highspeed, and steady. In addition, the security issues of mobile terminals and the Internet access are attached importance to. And as a combination of cloud computing, mobile devices and wireless networks, mobile cloud computing is an emerging but very promising paradigm which brings rich computational resources to mobile users, network operators, as well as cloud computing providers. The flaws of data storing and data computing in mobile-Internet applications can be overcome by mobile cloud computing while the new paradigm can also accomplish cloud based multi-user data sharing, end geographical service limitation, and process real-time tasks efficiently at the same time.

1.1. OBJECTIVES OF THE PROJECT

There is no accurate definition of mobile cloud computing, several concepts were proposed, and two most popular schemes can be described as follows: 1) Mobile cloud computing is a kind of scheme which could run an application such as a weather monitor application on remote cloud servers as displayed, while the mobile devices just act like normal PCs except that the mobile devices connect to cloud servers via 3G or 4G while PCs through Internet. 2) Taking advantages of leisure resources such as CPU, memory, and storing disks, another model of mobile cloud computing exploits the mobile devices themselves as resources providers of cloud. And the scheme supports user mobility, and recognizes the potential of mobile clouds to do collective sensing as well. In this Project, we mainly use the first paradigm mentioned above, but the second one inspires us to assume that what if the mobile devices do not provide computing resources or storing resources but sensing data instead? In fact, most mobile devices are capable to capture some data from the environment nowadays, for example, almost every smart phone are equipped with sensors of proximity, accelerometer, gyroscope, compass, barometer, camera, GPS,

microphone, etc. Combining the concept of WSN, mobile devices can be regarded as mobile sensors that are able to provide other mobile devices who are users of the mobile cloud services with some sensing information including environment monitoring data, health monitoring data, and so on. We take a weather monitor application as an example in this Project.

1.2. PROJECT DESCRIPTION

In this Project, a hierarchical access control method using a modified hierarchical attribute-based encryption (M-HABE) and a modified three-layer structure is proposed. Differing from the existing paradigms such as the HABE algorithm and the original three-layer structure, the novel scheme mainly focuses on the data processing, storing and accessing, which is designed to ensure the application users with legal access authorities to get corresponding sensing data and to restrict illegal users and unauthorized legal users get access to the data, the proposed promising paradigm makes it extremely suitable for the mobile cloud computing based paradigm. What should be emphasized is that the most important highlight of all in the proposed Project can be described as that the modified three-layer structure is designed for solving the security issues illustrated above

2. LITERATURE SURVEY

A literature survey or literature review means study of references Projects and old algorithms that we have read for designing the proposed methods. It also helps in reporting summarization of all the old references Projects, their drawbacks. The detailed literature survey for the project helps in comparing and contrasting various methods, algorithms in various ways that have implemented in the research.

2.1. RELATED WORK

1) Mobile cloud computing: A survey AUTHORS: N. Fernando, S. W. Loke, and W. Rahayu

Despite increasing usage of mobile computing, exploiting its full potential is difficult due to its inherent problems such as resource scarcity, frequent disconnections, and mobility. Mobile cloud computing can address these problems by executing mobile applications on resource providers external to the mobile device. In this Project, we provide an extensive survey of mobile cloud computing research, while highlighting the specific concerns in mobile cloud computing. We present a taxonomy based on the key issues in this area, and discuss the different approaches taken to tackle these issues. We conclude the Project with a critical analysis of challenges that have not yet been fully met, and highlight directions for future work.

2) Cloud-based augmentation for mobile devices: motivation, taxonomies, and open challenges AUTHORS: S. Abolfazli, Z. Sanaei, E. Ahmed, A. Gani, and R. Buyya

Recently, Cloud-based Mobile Augmentation (CMA) approaches have gained remarkable ground from academia and industry. CMA is the state-of-the-art mobile augmentation model that employs resource-rich clouds to increase, enhance, and optimize computing capabilities of mobile devices aiming at execution of resource-intensive mobile applications. Augmented mobile devices envision to perform extensive computations and to store big data beyond their intrinsic capabilities with least footprint and vulnerability. Researchers utilize varied cloud-based computing resources (e.g., distant clouds and nearby mobile nodes) to meet various computing

requirements of mobile users. However, employing cloud-based computing resources is not a straightforward panacea. Comprehending critical factors (e.g., current state of mobile client and remote resources) that impact on augmentation process and optimum selection of cloud-based resource types are some challenges that hinder CMA adaptability.

3) Mobile cloud computing: Standard approach to protecting and securing of mobile cloud ecosystems **AUTHORS:** R. Kumar and S. Rajalakshmi

The concepts of Cloud computing are naturally meshed with mobile devices to enable on-the-go functionalities and benefits. The mobile cloud computing is emerging as one of the most important branches of cloud computing and it is expected to expand the mobile ecosystems. As more mobile devices enter the market and evolve, certainly security issues will grow as well. Also, enormous growth in the variety of devices connected to the Internet will further drive security needs. Understanding the true potential of mobile cloud computing and identifying issues with mobile cloud security, privacy, feasibility and accessibility remain a major concern for both the customers and the enterprises.

4) Mobile cloud sensing, big data, and 5g networks make an intelligent and smart world **AUTHORS:** Q. Han, S. Liang, and H. Zhang

Through time, we have seen mobile phones transform into multifaceted devices, adapted to meet and exceed our everyday needs. These needs range from something as personal as a health care manager to something as purely analytical as an environment monitor. In effect, mobile phones have come into our lives, making life easier, smarter, and more efficient. In this article we discuss mobile sensing and cloud computing separately and in detail, then combine the two concepts to form the singular idea of mobile cloud sensing. We will also give an intuitive architectural description of mobile cloud sensing, along with discussions about each of its individual building blocks. There are limitations to mobile cloud sensing today, but with the emergence of 5G coupled with the analysis of big data, we can address the current issues at hand.

3. SYSTEM ANALYSIS

3.1. EXISTING SYSTEM

Senders encrypt message with certain attributes of the authorized receivers. The ABE based access control method uses several tags to mark the attributes that a specific authorized user needs to possess. The users with certain tag sets can get access to the specific encrypted data and decrypt it. Lots of Project introduced the scheme about the attribute based encryption access control method in the cloud computing. In the mobile loud computing environment, there are tremendous data which needs to be processed and marked with attributions for the convenient attributing access before storing. At the same time, the hierarchical structure of the application users need an authentication center entity to control their attributes.

3.1.1. Disadvantages

- Does not guarantee Availability
- Issues of Confidentiality. Consumers' data were not kept secret in cloud systems
- Data Integrity Issue
- No Multiple Controls

3.2. PROPOSED SYSTEM

In the proposed scenario, users with different privilege levels have different rights to access the part of sensing data coming from the mobile devices. Therefore, one same data has to be encrypted into ciphertext once, which ought to be able to be decrypted multiple times by different authorized users. In this Project, a hierarchical access control method using a modified hierarchical attribute-based encryption (M-HABE) and a modified three-layer structure is proposed. Differing from the existing paradigms such as the HABE algorithm and the original three-layer structure, the novel scheme mainly focuses on the data processing, storing and accessing, which is designed to ensure the application users with legal access authorities to get corresponding sensing data and to restrict illegal users and unauthorized legal users get access to the data, the proposed promising paradigm makes it extremely suitable for the mobile cloud computing based paradigm. What should be emphasized is that

the most important highlight of all in the proposed Project can be described as that the modified three-layer structure is designed for solving the security issues illustrated above.

3.2.1. Advantages

- One ciphertext can be decrypted by several keys.
- Both precise level description and user attribute should be supported in the access structure of the method.
- The keys in the authentication center ought to have the same hierarchical structure just as the structure of users privilege levels.

3.3. FEASIBILITY STUDY

All projects are feasible when provided with unlimited resources and infinite time. Unfortunately, the development of computer-based system or product is more likely plagued by a scarcity of resources and difficult delivery dates. It is both necessary and prudent to evaluate the feasibility of a project at the earliest possible time. Months or years of effort, thousands or millions of dollars, and untold professional embarrassment can be averted if an ill-conceived system is recognized early in the definition phase.

Feasibility and risk analysis are related in many ways. If project risk is great the feasibility of producing quality software is reduced. During product engineering, however, we concentrate our attention on four primary areas of interest.

3.3.1. Technical Feasibility

This application is going to be used in an Internet environment called www (World Wide Web). So, it is necessary to use a technology that is capable of providing the networking facility to the application. This application is also able to work on distributed environment. Application is developed with DOT NET Technology. One major advantage in application is platform neutral. We can deploy and use it in any operating system.

GUI is developed using HTML to capture the information from the customer. HTML is used to display the content on the browser. It uses TCP/IP protocol. It is an interpreted language. It is very easy to develop a page/document using HTML some

RAD (Rapid Application Development) tools are provided to quickly design/develop our application. So many objects such as button, text fields, and text area etc are provided to capture the information from the customer.

3.3.2. Economical Feasibility

The economical issues usually arise during the economical feasibility stage are whether the system will be used if it is developed and implemented, whether the financial benefits are equal or exceeds the costs. The cost for developing the project will include cost conducts full system investigation, cost of hardware and software for the class of being considered, the benefits in the form of reduced costs or fewer costly errors.

The project is economically feasible if it is developed and installed. It reduces the work load. Keep the class of application in the view, the cost of hardware and software is considered to be economically feasible.

3.3.3. Operational Feasibility

In our application front end is developed using GUI. So it is very easy to the customer to enter the necessary information. But customer must have some knowledge on using web applications before going to use our application.

3.4. Modules and its Description

- User
- Authentication
- Sub Authentication 1
- Sub Authentication 2

3.4.1. User Module

In this module users first register based on his Level and login his Account. User can able to upload a file to cloud for some level of users. Any level user can upload file for any level of users. Users can only view their level file. users can also download the file but users give private key requests to Sub Authentication1 and file decryption key requests to Sub Authentication 2. Then users download their level files by using private and decryption key.

3.4.2. Authentication

In this module can able to view User Details and Uploaded file details. Authentication can able view the private key generated files by Sub Authentication 1 and able to view the decryption key generated files by Sub Authentication 2.

3.4.3. Sub Authentication 1

In this module the Sub Authentication 1 can able to view the user details and response the users private key requests. Sub Authentication 1 send the private key to the requested user then only the user can download the file.

3.4.4. Sub Authentication 2

In this module the Sub Authentication 2 can able to view the user details and response the users private key requests. Sub Authentication 2 send the file decryption key to the requested user then only the user can download the file.

4. SOFTWARE / HARDWARE REQUIREMENT SPECIFICATIONS

4.1. SOFTWARE REQUIREMENTS

Operating System	:	Windows XP/7/8
Front End	:	JSP
Database	:	MySql
Programming language	:	Java

4.2. HARDWARE REQUIREMENTS

Processor	:	Pentium Dual Core/ Core to Duo/ ICore with Minimum 1.2 GHZ Speed
RAM	:	1 GB
Hard Disk	:	120 GB

5. SYSTEM DESIGN

5.1. UML DIAGRAMS INTRODUCTION

The unified modeling language allows the software engineer to express an analysis model using the modeling notation that is governed by a set of syntactic, semantic and pragmatic rules. A UML system is represented using five different views that describe the system from distinctly different perspective.

UML is specifically constructed through two different domains they are:

- UML Analysis modeling, this focuses on the user model and structural model views of the system.
- UML design modeling, which focuses on the behavioral modeling, implementation modeling and environmental model views.

5.2. SYSTEM DESIGN ASPECTS

Once the analysis stage is completed, the next stage is to determine in broad outline form how the problem might be solved. During system design, we are beginning to move from the logical to physical level.

System design involves architectural and detailed design of the system. Architectural design involves identifying software components, decomposing them into processing modules and conceptual data structures, and specifying the interconnections among components.

Detailed design is concerned with how to package processing modules and how to implement the processing algorithms, data structures and interconnections of standard algorithms, invention of new algorithms, and design of data representations and packaging of software products. Two kinds of approaches are available:

- Top down approach
- Bottom up approach

5.2.1. Design of Code

Since information systems projects are designed with space, time and cost saving in mind, coding methods in which conditions, words, ideas or control errors and speed the entire process. The purpose of the code is to facilitate the identification and retrieval of the information. A code is an ordered collection of symbols designed to provide unique identification of an entity or an attribute.

5.2.2. Design of Input

Design of input involves the following decisions

- Input data
- Input medium
- The way data should be arranged or coded
- Validation needed to detect every step to follow when error occurs

The input controls provide ways to ensure that only authorized users access the system guarantee the valid transactions, validate the data for accuracy and determine whether any necessary data has been omitted. The primary input medium chosen is display. Screens have been developed for input of data using HTML. The validations for all important inputs are taken care of through various events using JSP control.

5.2.3. Design of Output

Design of output involves the following decisions

- Information to present
- Output medium
- Output layout

Output of this system is given in easily understandable, user-friendly manner, Layout of the output is decided through the discussions with the different users.

5.2.4. Design of Control

The system should offer the means of detecting and handling errors.

Input controls provides ways to

- Valid transactions are only acceptable
- Validates the accuracy of data
- Ensures that all mandatory data have been captured

All entities to the system will be validated. And updating of tables is allowed for only valid entries. Means have been provided to correct, if any by change incorrect entries have been entered into the system they can be edited.

5.3. UML DIAGRAMS

Why we have to use UML in projects?

As the strategic value of software increases for many companies, the industry looks for techniques to automate the production of software and to improve quality and reduce cost and time-to-market. These techniques include component technology, visual programming, patterns and frameworks. Businesses also seek techniques to manage the complexity of systems as they increase in scope and scale. In particular, they recognize the need to solve recurring architectural problems, such as physical distribution, concurrency, replication, security, load balancing and fault tolerance. Additionally, the development for the World Wide Web, while making some things simpler, has exacerbated these architectural problems. The Unified Modeling Language (UML) was designed to respond to these needs. Simply, Systems design refers to the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements which can be done easily through UML diagrams.

In the project eight basic UML diagrams have been explained

- Class Diagram
- Use Case Diagram
- Sequence Diagram
- Activity Diagram
- Collaboration Diagram
- Deployment Diagram
- State Chart Diagram
- Component Diagram

5.3.1. Class Diagram

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, and the relationships between the classes.

This is one of the most important of the diagrams in development. The diagram breaks the class into three layers. One has the name, the second describes its attributes and the third its methods. A padlock to left of the name represents the private attributes. The relationships are drawn between the classes. Developers use the Class Diagram to develop the classes. Analyses use it to show the details of the system.

Architects look at class diagrams to see if any class has too many functions and see if they are required to be split.

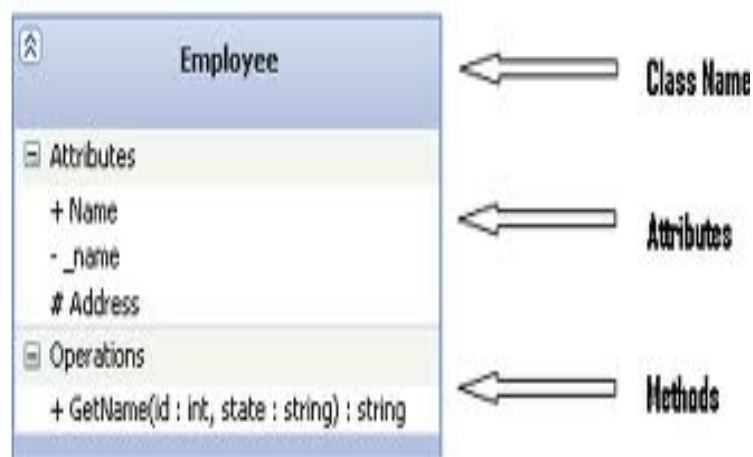


Fig 5.1. Class Diagram

5.3.2. Use Case Diagram

In software engineering, a use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in

the system can be depicted. Use cases are used during requirements elicitation and analysis to represent the functionality of the system. Use cases focus on the behavior of the system from the external point of view. The actors are outside the boundary of the system, whereas the use cases are inside the boundary of the system

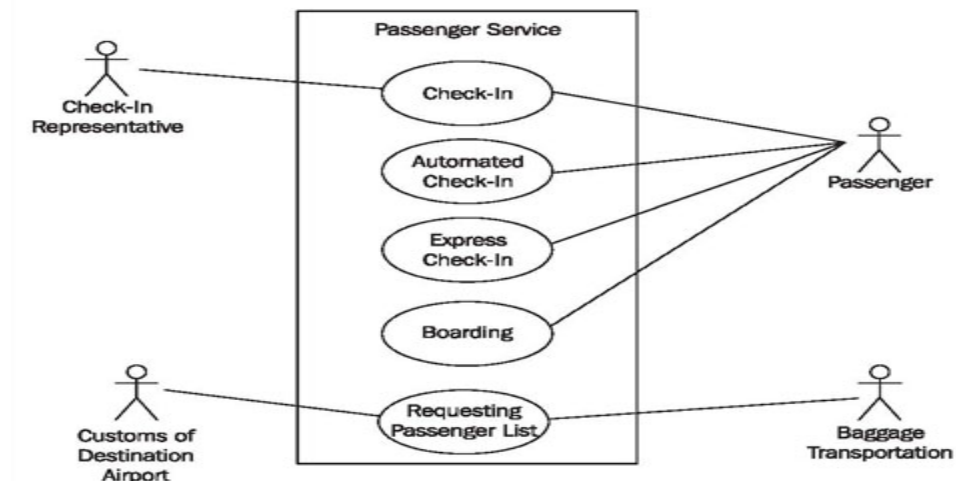


Fig 5.2. Use Case Diagram

5.3.3. Sequence Diagram

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called Event-trace diagrams, event scenarios, and timing diagrams

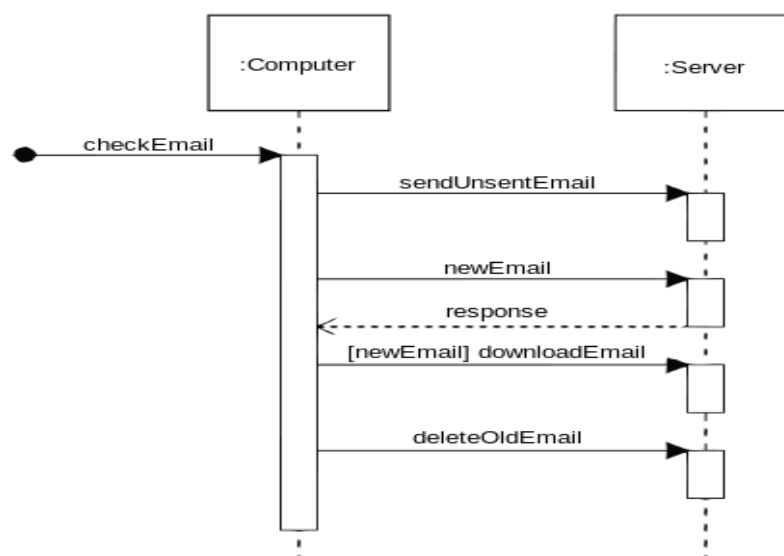


Fig 5.3. Sequence Diagram

5.3.4. Activity Diagram

Activity diagrams are a loosely defined diagram technique for showing workflows of stepwise activities and actions, with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

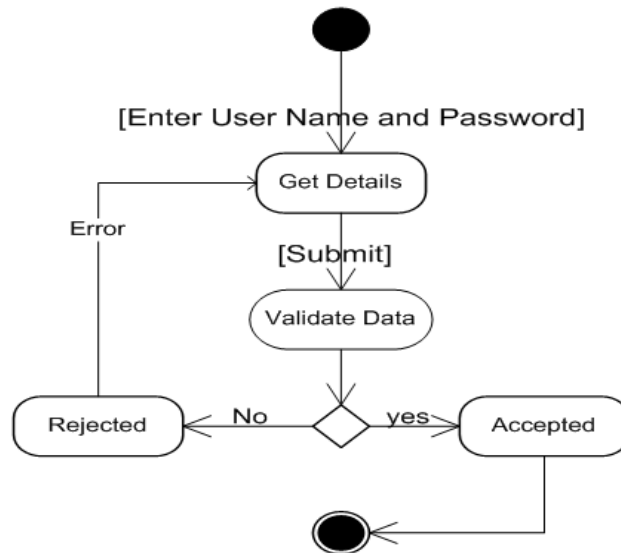


Fig 5.4. Activity Diagram

5.3.5. Collaboration Diagram

A Communication diagram models the interactions between objects or parts in terms of sequenced messages. Communication diagrams represent a combination of information taken from Class, Sequence, and Use Case Diagrams describing both the static structure and dynamic behavior of a system.

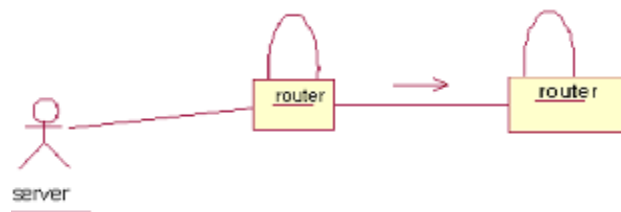


Fig 5.5. Collaboration Diagram

5.3.6. Deployment Diagram

A deployment diagram in the Unified Modeling Language models the physical deployment of artifacts on nodes. To describe a web site, for example, a deployment diagram would show what hardware components ("nodes") exist (e.g., a web server, an application server, and a database server), what software components ("artifacts") run on each node (e.g., web application, database), and how the different pieces are connected e.g. JDBC, REST

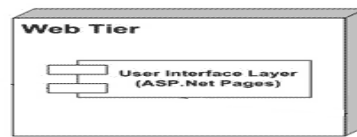


Fig 5.6. Deployment Diagram

5.3.7. State Chart Diagram

A state diagram is a type of diagram used in computer science and related fields to describe the behavior of systems. State diagrams require that the system described is composed of a finite number of states sometimes, this is indeed the case, while at other times this is a reasonable abstraction. Many forms of state diagrams exist, which differ slightly and have different semantics.

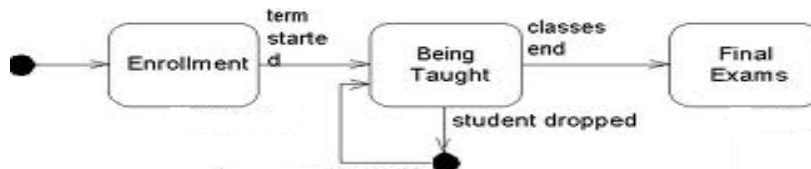


Fig 5.7. State Chart Diagram

5.3.8. Component Diagram

In the Unified Modeling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems.

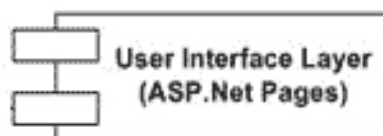


Fig 5.8. Component Diagram

CLASS DIAGRAM

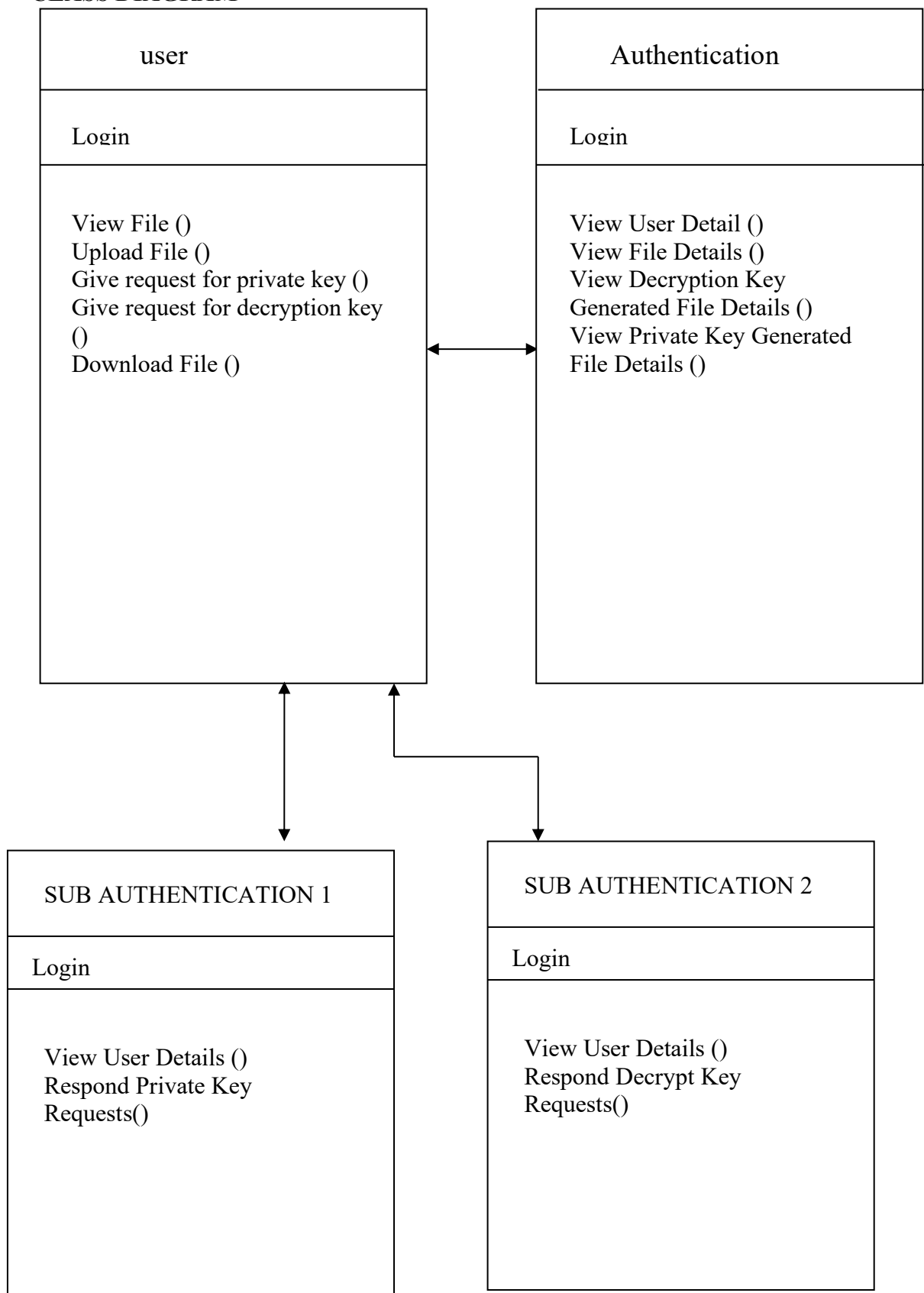


Fig 5.9. Class Diagram for Overall Project

USE CASE DIAGRAM

A Use case diagram shows a set of use cases and actors and their relationships. Use case diagrams address the static Use case view of a system. These diagrams are especially important in organizing and modeling the behaviors of a system.

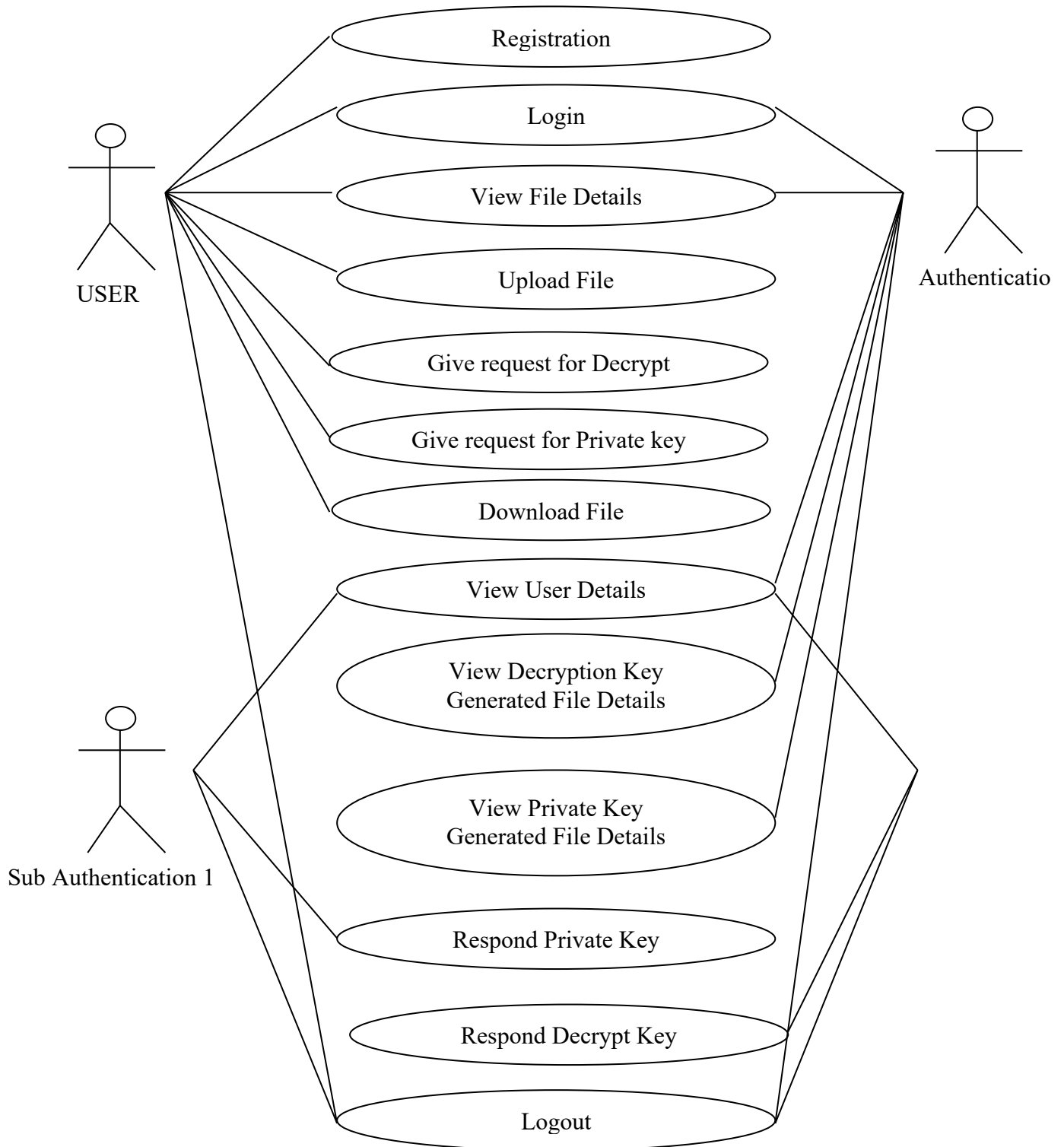


Fig. 5.10. Use Case Diagram for Administrator

5.3.9. Sequence Diagram

The sequence diagram is an interaction diagram that emphasizes the time ordering of messages for modeling a real time system. Graphically, a sequence diagram is a table that shows objects arranged along the X axis and messages, ordered in increasing time, along the Y axis. Sequence Diagram consists of objects, links, lifeline, focus of control, and messages.

- **OBJECT:** Objects are typically named or anonymous instances of class. But may also represent instances of other things such as components, collaboration and nodes.
- **LINK:** A link is a semantic connection among objects i.e., an object of an association is called as a link.
- **LIFELINE:** A lifeline is vertical dashed line that represents the lifetime of an object.
- **FOCUS OF CONTROL:** A Focus of Control is tall, thin rectangle that shows the period of time during which an object is performing an action.
- **MESSAGES:** A message is a specification of a communication between objects that conveys the information with the expectation that the activity will ensue.

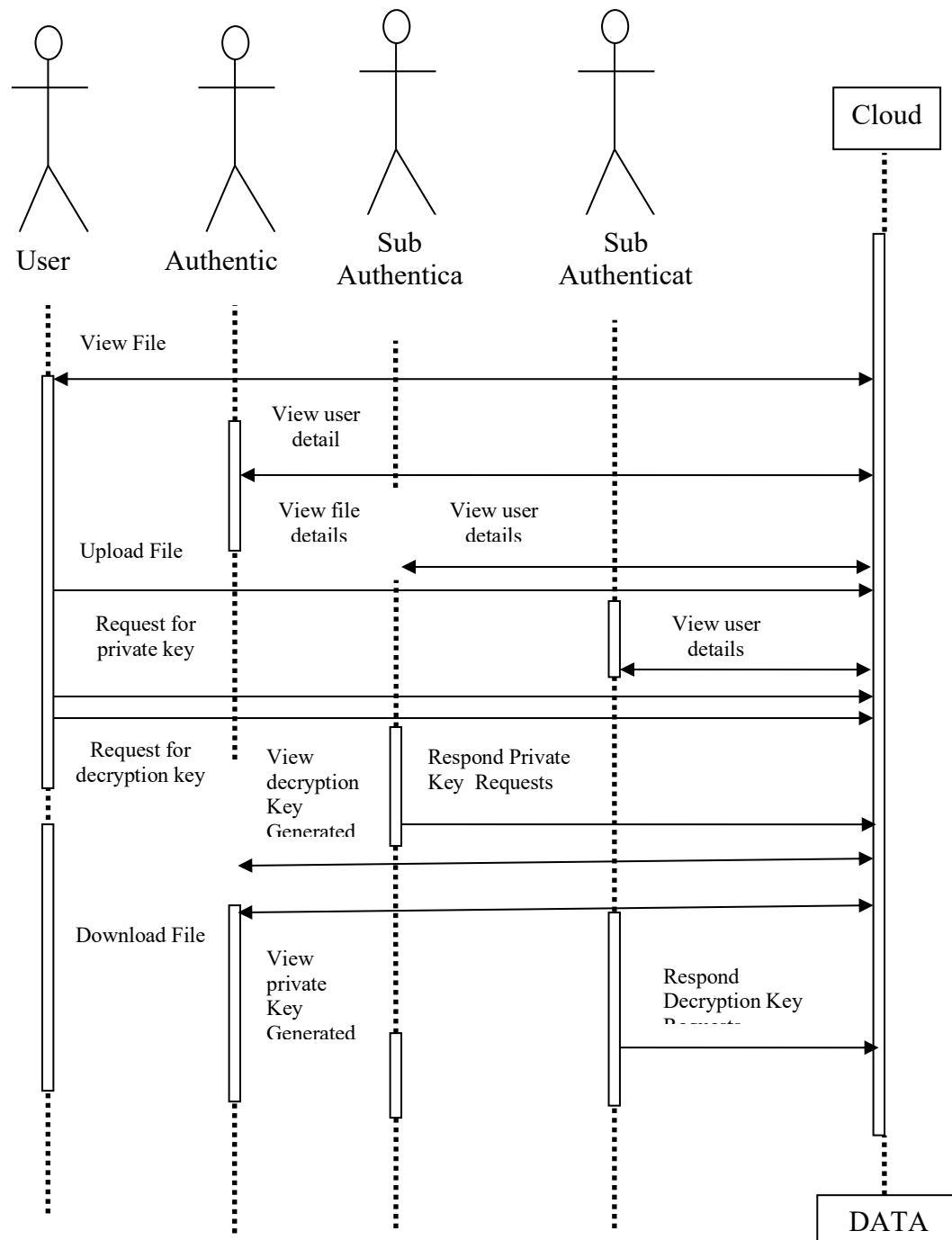


Fig. 5.11. Sequence Diagram

5.3.10. Activity Diagram

An activity diagram shows the flow from activity to activity. The activity diagram emphasizes the dynamic view of a system. It consists of activity states, action states, transition, and object.

- **ACTIVITY STATE:** An activity states is a kind of states in activity diagram; it shows an ongoing non-atomic execution within a state machine. An activity states can be further decomposed.
- **ACTION STATE:** An action states are states of the system, each representing the execution of an action. An action states can't be further decomposed.
- **TRANSITION:** A transition specifies the path from one action or activity state to the next action or activity state. The transition is rendered as a simple directed line.
- **OBJECT:** An object is a concrete manifestation of an abstraction; an entity with a well defined boundary and identity that encapsulates state and behavior; an instance of a class. Objects may be involved in the flow of control associated with an activity diagram.

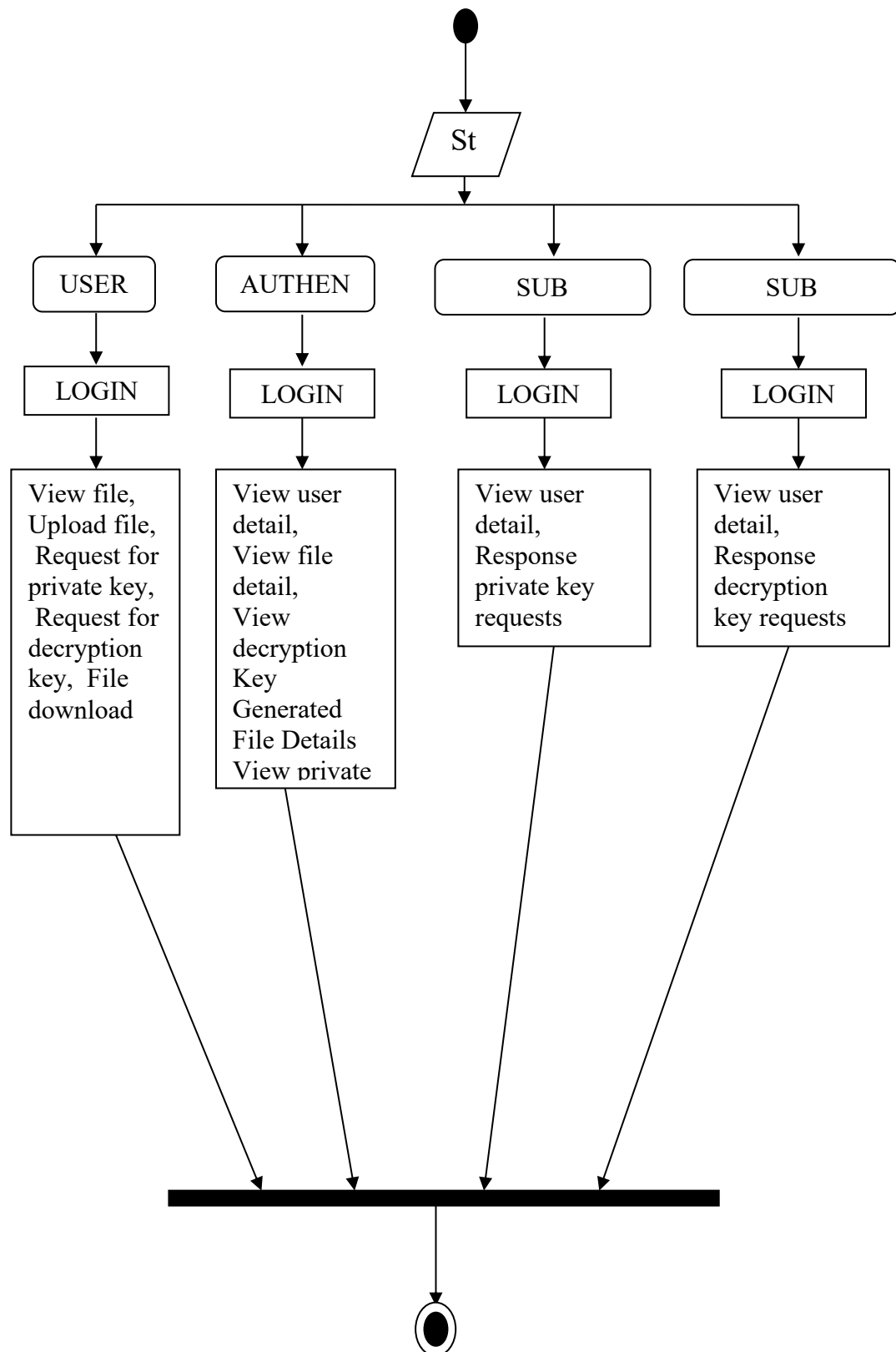


Fig 5.12. Activity Diagram for Administrator

5.3.11. State Transition Diagram

What is State Transition Testing?

State Transition testing, a black box testing technique, in which outputs are triggered by changes to the input conditions or changes to 'state' of the system. In other words, tests are designed to execute valid and invalid state transitions.

When to use?

- When we have sequence of events that occur and associated conditions that apply to those events
- When the proper handling of a particular event depends on the events and conditions that have occurred in the past
- It is used for real time systems with various states and transitions involved

Deriving Test cases:

- Understand the various state and transition and mark each valid and invalid state
- Defining a sequence of an event that leads to an allowed test ending state
- Each one of those visited state and traversed transition should be noted down
- Steps 2 and 3 should be repeated until all states have been visited and all transitions traversed
- For test cases to have a good coverage, actual input values and the actual output values have to be generated

Advantages:

- Allows testers to familiarise with the software design and enables them to design tests effectively.
- It also enables testers to cover the unplanned or invalid states.

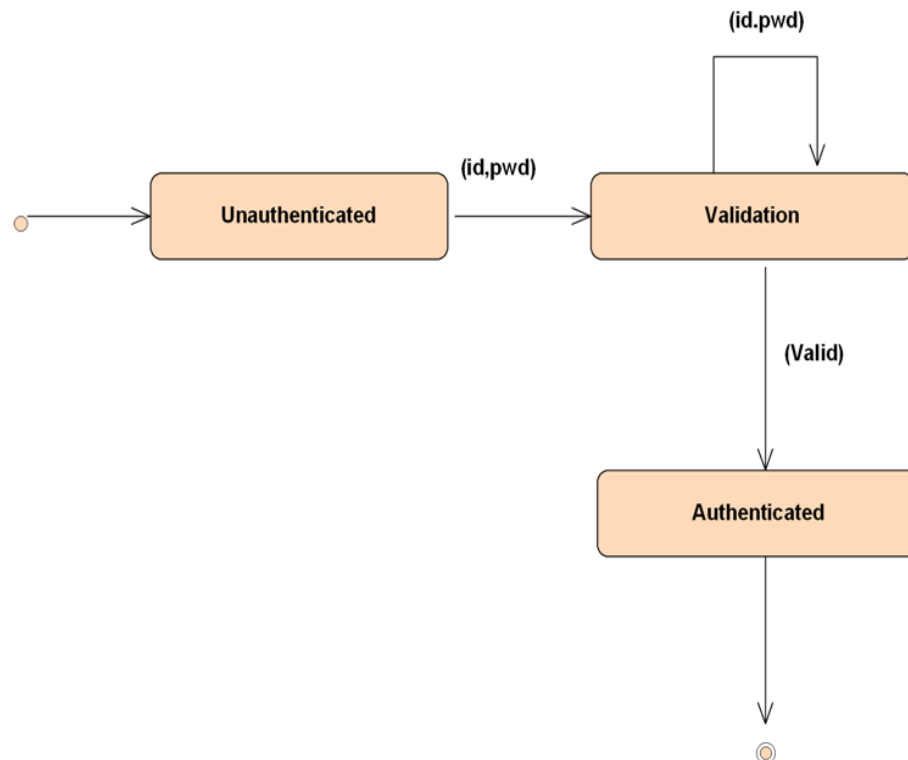


Fig. 5.13.State Transition Diagram for Manager

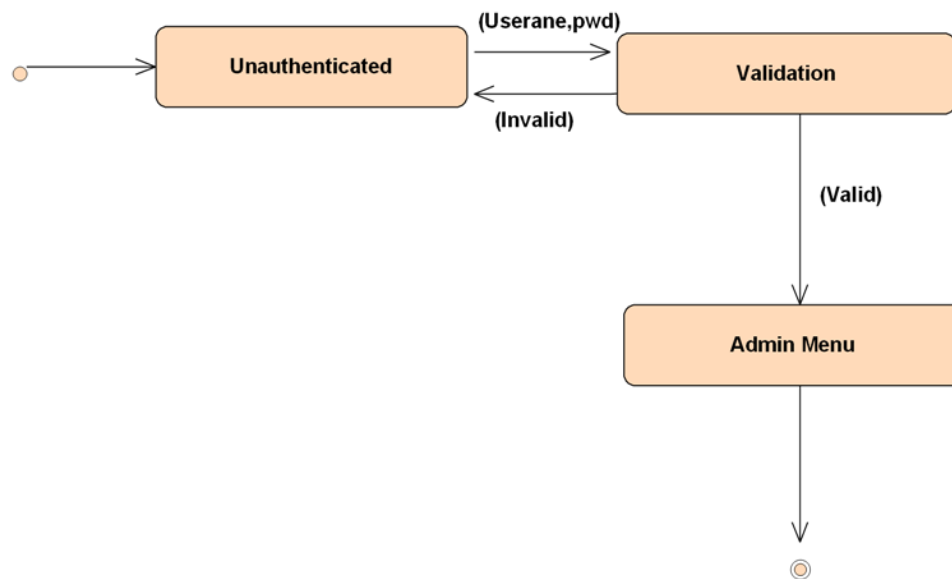


Fig. 5.14.State Transition Diagram for Employee

5.3.12. DATA FLOW DIAGRAMS

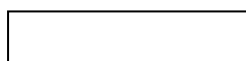
Data drive business activities. They can trigger events and processed to provide information useful to personnel. System analysis recognizes the central role of business data in the organization. The following that shows the flow of data through business processes, which is the purpose of data flow analysis, tells us a great deal about how organization objectives are accomplished.

Data flow analysis studies the use of data in each activity. It documents these details in the Data Flow Diagrams.

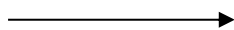
A data flow diagram is a logical model of a system. The model does not depend on hardware, software, and data structures of the organization. There is no physical implication in a data flow diagram because the diagram is a graphic picture of the logical system, it tends to be easy for every non-technical user to understand and thus serves as an excellent communication tool. Finally a data flow diagram is a good starting point for system design.

To construct a data flow diagram it uses basic symbols. They are given below.

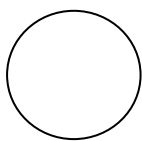
DFD NOTATIONS



Define source and destination data.



Shows path of the data flow.



To represent a process that transforms or
modifies the Data



To represent an attribute



Data Store

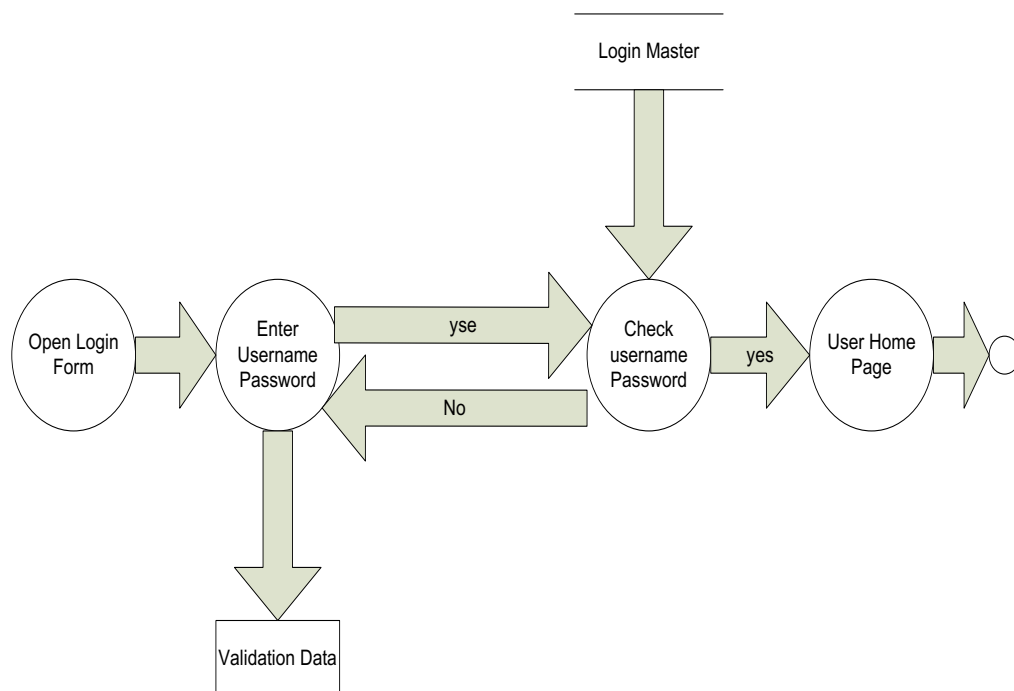


Fig 5.15. Data Flow Diagram for Home Page

Context Level DFD

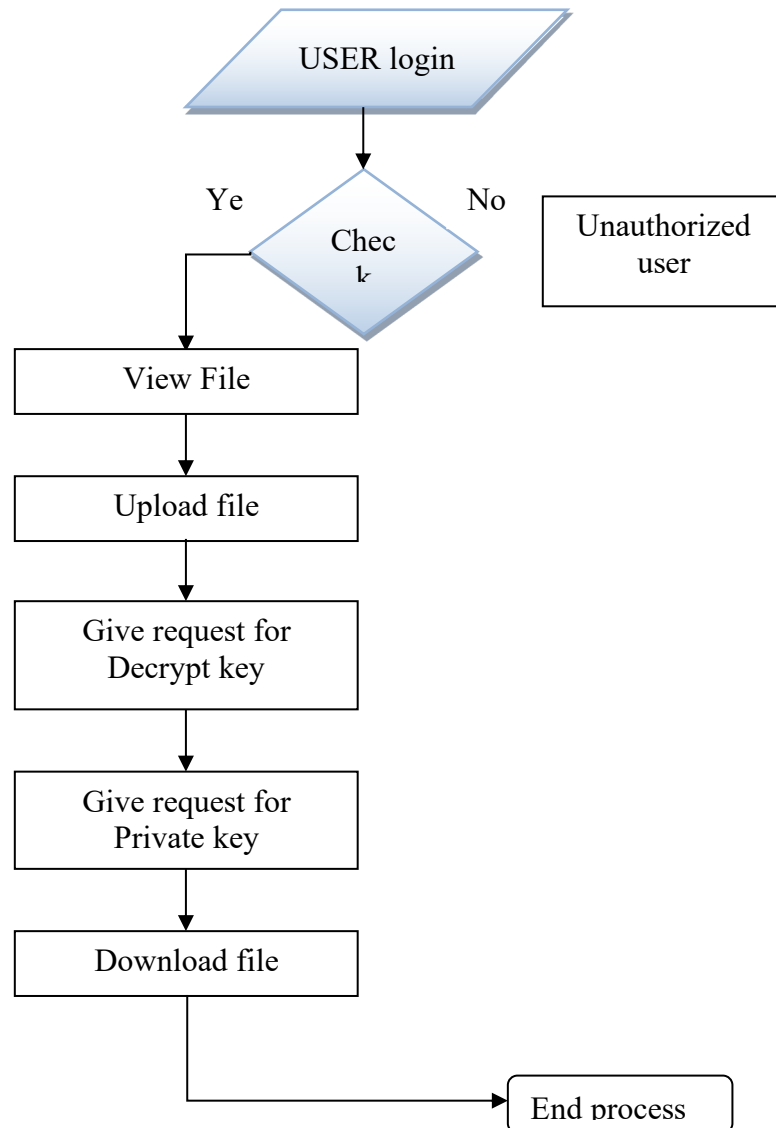


Fig. 5.16. Context Level Data Flow Diagram

6. IMPLEMENTATION

6.1. JAVA TECHNOLOGY

Java technology is both a programming language and a platform.

The Java Programming Language:

The Java programming language is a high-level language that can be characterized by all of the following buzzwords:

- Simple
- Architecture neutral
- Object oriented
- Portable
- Distributed
- High performance
- Interpreted
- Multithreaded
- Robust
- Dynamic
- Secure

With most programming languages, you either compile or interpret a program so that you can run it on your computer. The Java programming language is unusual in that a program is both compiled and interpreted. With the compiler, first you translate a program into an intermediate language called Java byte codes —the platform-independent codes interpreted by the interpreter on the Java platform. The interpreter parses and runs each Java byte code instruction on the computer. Compilation happens just once; interpretation occurs each time the program is executed. The following figure illustrates how this works.

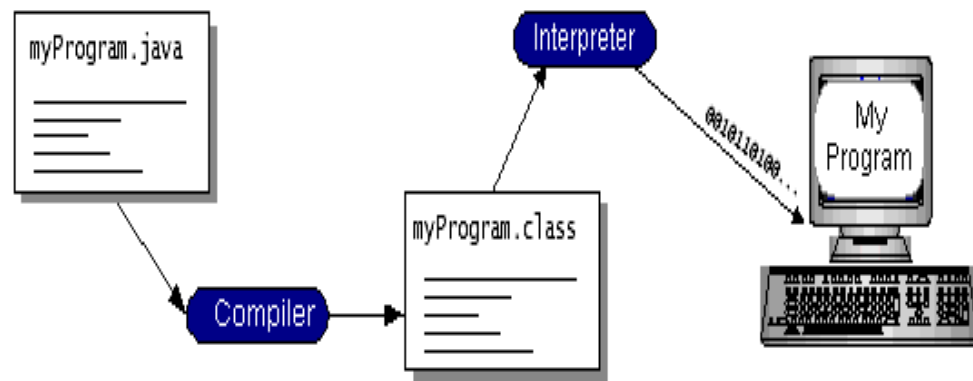


Fig 6.1. Working of Java Program

If we think of Java byte codes as the machine code instructions for the Java Virtual Machine (Java VM). Every Java interpreter, whether it's a development tool or a Web browser that can run applets, is an implementation of the Java VM. Java byte codes help make “write once, run anywhere” possible. You can compile your program into byte codes on any platform that has a Java compiler. The byte codes can then be run on any implementation of the Java VM. That means that as long as a computer has a Java VM, the same program written in the Java programming language can run on Windows 2000, a Solaris workstation, or on an iMac.

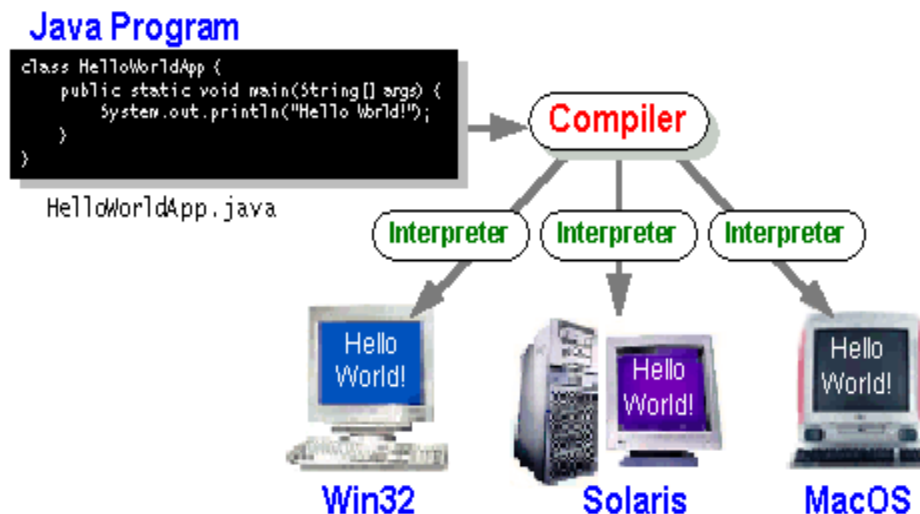


Fig 6.2. Implementation of Java Virtual Machine

6.2. THE JAVA PLATFORM

A platform is the hardware or software environment in which a program runs. We've already mentioned some of the most popular platforms like Windows 2000, Linux, Solaris, and MacOS. Most platforms can be described as a combination of the

operating system and hardware. The Java platform differs from most other platforms in that it's a software-only platform that runs on top of other hardware-based platforms.

The Java platform has two components:

- The Java Virtual Machine (Java VM)
- The Java Application Programming Interface (Java API)

You've already been introduced to the Java VM. It's the base for the Java platform and is ported onto various hardware-based platforms.

The Java API is a large collection of ready-made software components that provide many useful capabilities, such as graphical user interface (GUI) widgets. The Java API is grouped into libraries of related classes and interfaces; these libraries are known as packages. In the next section, What Can Java Technology Do? Highlights what functionality some of the packages in the Java API provide.

The following figure depicts a program that's running on the Java platform. As the figure shows, the Java API and the virtual machine insulate the program from the hardware.

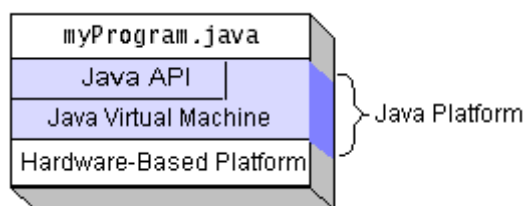


Fig 6.3. Program Running on the Java Platform

Native code is code that after you compile it, the compiled code runs on a specific hardware platform. As a platform-independent environment, the Java platform can be a bit slower than native code. However, smart compilers, well-tuned interpreters, and just-in-time byte code compilers can bring performance close to that of native code without threatening portability.

What Can Java Technology Do?

The most common types of programs written in the Java programming language are applets and applications. If you've surfed the Web, you're probably already familiar with applets. An applet is a program that adheres to certain conventions that allow it to run within a Java-enabled browser.

However, the Java programming language is not just for writing cute, entertaining applets for the Web. The general-purpose, high-level Java programming language is also a powerful software platform. Using the generous API, you can write many types of programs.

An application is a standalone program that runs directly on the Java platform. A special kind of application known as a server serves and supports clients on a network. Examples of servers are Web servers, proxy servers, mail servers, and print servers. Another specialized program is a Servlet. A Servlet can almost be thought of as an applet that runs on the server side. Java Servlets are a popular choice for building interactive web applications, replacing the use of CGI scripts. Servlets are similar to applets in that they are runtime extensions of applications. Instead of working in browsers, though, Servlets run within Java Web servers, configuring or tailoring the server.

How does the API support all these kinds of programs? It does so with packages of software components that provides a wide range of functionality. Every full implementation of the Java platform gives you the following features:

- **The essentials:** Objects, strings, threads, numbers, input and output, data structures, system properties, date and time, and so on.
- **Applets:** The set of conventions used by applets.
- **Networking:** URLs, TCP (Transmission Control Protocol), UDP (User Data gram Protocol) sockets, and IP (Internet Protocol) addresses.
- **Internationalization:** Help for writing programs that can be localized for users worldwide. Programs can automatically adapt to specific locales and be displayed in the appropriate language.

- **Security:** Both low level and high level, including electronic signatures, public and private key management, access control, and certificates.
- **Software components:** Known as JavaBeansTM, can plug into existing component architectures.
- **Object serialization:** Allows lightweight persistence and communication via Remote Method Invocation (RMI).
- **Java Database Connectivity (JDBCTM):** Provides uniform access to a wide range of relational databases.

The Java platform also has APIs for 2D and 3D graphics, accessibility, servers, collaboration, telephony, speech, animation, and more. The following figure depicts what is included in the Java 2 SDK.

6.3. ODBC

Microsoft Open Database Connectivity (ODBC) is a standard programming interface for application developers and database systems providers. Before ODBC became a de facto standard for Windows programs to interface with database systems, programmers had to use proprietary languages for each database they wanted to connect to. Now, ODBC has made the choice of the database system almost irrelevant from a coding perspective, which is as it should be. Application developers have much more important things to worry about than the syntax that is needed to port their program from one database to another when business needs suddenly change.

Through the ODBC Administrator in Control Panel, you can specify the particular database that is associated with a data source that an ODBC application program is written to use. Think of an ODBC data source as a door with a name on it. Each door will lead you to a particular database. For example, the data source named Sales Figures might be a SQL Server database, whereas the Accounts Payable data source could refer to an Access database. The physical database referred to by a data source can reside anywhere on the LAN.

The ODBC system files are not installed on your system by Windows 95. Rather, they are installed when you setup a separate database application, such as SQL Server Client or Visual Basic 4.0. When the ODBC icon is installed in Control Panel, it uses a file called ODBCINST.DLL. It is also possible to administer your ODBC data sources through a stand-alone program called ODBCADM.EXE. There is a 16-bit and a 32-bit version of this program and each maintains a separate list of ODBC data sources.

6.4. JDBC

In an effort to set an independent database standard API for Java; Sun Microsystems developed Java Database Connectivity, or JDBC. JDBC offers a generic SQL database access mechanism that provides a consistent interface to a variety of RDBMSs. This consistent interface is achieved through the use of “plug-in” database connectivity modules, or drivers. If a database vendor wishes to have JDBC support, he or she must provide the driver for each platform that the database and Java run on.

To gain a wider acceptance of JDBC, Sun based JDBC’s framework on ODBC. As you discovered earlier in this chapter, ODBC has widespread support on a variety of platforms. Basing JDBC on ODBC will allow vendors to bring JDBC drivers to market much faster than developing a completely new connectivity solution.

JDBC was announced in March of 1996. It was released for a 90 day public review that ended June 8, 1996. Because of user input, the final JDBC v1.0 specification was released soon after.

The remainder of this section will cover enough information about JDBC for you to know what it is about and how to use it effectively. This is by no means a complete overview of JDBC. That would fill an entire book.

7. CODING

```
<%--
```

```
Document : index
```

```
Created on : Dec 22, 2016, 3:34:13 PM
```

```
Author : java3
```

```
--%>
```

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
```

```
<!-- === BEGIN HEADER === -->
```

```
<!DOCTYPE html>
```

```
<!--[if IE 8]> <html lang="en" class="ie8"> <![endif]-->
```

```
<!--[if IE 9]> <html lang="en" class="ie9"> <![endif]-->
```

```
<!--[if !IE]><!-->
```

```
<html lang="en">
```

```
<!--<![endif]-->
```

```
<head>
```

```
<title>A Modified Hierarchical Attribute-Based Encryption</title>
```

```
<!-- Meta -->
```

```
<meta http-equiv="content-type" content="text/html; charset=utf-8" />
```

```
<meta name="description" content="">
```

```
<meta name="author" content="">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1,  
maximum-scale=1" />
```

```
<!-- Favicon -->
```

```
<link href="favicon.ico" rel="shortcut icon">
```

```
<!-- Bootstrap Core CSS -->
```

```
<link rel="stylesheet" href="assets/css/bootstrap.css" rel="stylesheet">
```

```
<!-- Template CSS -->

<link rel="stylesheet" href="assets/css/animate.css" rel="stylesheet">

<link rel="stylesheet" href="assets/css/font-awesome.css" rel="stylesheet">

<link rel="stylesheet" href="assets/css/nexus.css" rel="stylesheet">

<link rel="stylesheet" href="assets/css/responsive.css" rel="stylesheet">

<link rel="stylesheet" href="assets/css/custom.css" rel="stylesheet">

<!-- Google Fonts-->

<link rel="stylesheet" type="text/css"
href="http://fonts.googleapis.com/css?family=Open+Sans:300,400" />

<link rel="stylesheet" type="text/css"
href="http://fonts.googleapis.com/css?family=Source+Sans+Pro:300,400" />

</head>

<body>

<div id="body_bg">

<div id="container_header" class="container">

<div id="header" class="row">

<div class="clear"><h1>A Modified Hierarchical Attribute-Based
Encryption Access Control Method for Mobile Cloud Computing</h1></div>

</div>

</div>

<div class="primary-container-group">

<!-- Background -->

<div class="primary-container-background">

<div class="primary-container"></div>

<div class="clearfix"></div>

</div>

<!--End Background -->
```

```
<div class="primary-container">

  <div id="container_hornav" class="container no-padding">

    <!-- Logo -->

    <div class="logo">

      <a href="index.html">

        </a>

      </div>

    <!-- End Logo -->

    <!-- Slogan -->

    <!-- End Slogan -->

    <!-- Top Menu -->

    <div class="row">

      <div class="hornav-block">

        <div id="hornav" class="pull-right">

          <ul id="hornavmenu" class="nav navbar-nav">

            <li>

              <a href="index.jsp">HOME</a>

            </li>

            <li>

              <span>USER</span>

              <ul>

                <li>

                  <a href="ulog.jsp">LOGIN</a>

                </li>

              </ul>

            </li>

          </ul>

        </div>

      </div>

    </div>

  </div>

</div>
```

```
<li>

    <a href="ureg.jsp">REGISTRATION</a>

</li>

</ul>

</li>

<li>

    <a href="authenlog.jsp">Authentication Center</a>

</li>

<li>

    <span>Sub Authentications</span>

    <ul>

        <li><a href="sauthen1log.jsp">Sub Authentication
1</a></li>

        <li><a href="sauthen1log.jsp">Sub Authentication
2</a></li>

    </ul>

</li>

</ul>

</div>

</div>

<div class="clearfix"></div>

</div>

<!-- End Top Menu -->

</div>

<div class="container no-padding">

    <!-- === END HEADER === -->
```

```
<!-- === BEGIN CONTENT === -->  
  
<div class="row">  
  
</body>  
  
</html>  
  
<!-- === END FOOTER === -->
```

8. TESTING

8.1. SOFTWARE TESTING TECHNIQUES

Software Testing is a critical element of software quality assurance and represents the ultimate review of specification, design and coding, Testing presents an interesting anomaly for the software engineer.

8.1.1. Testing Objectives

1. Testing is a process of executing a program with the intent of finding an error.
2. A good test case is one that has a probability of finding an as yet undiscovered error.
3. A successful test is one that uncovers an undiscovered error.

These above objectives imply a dramatic change in view port.

Testing cannot show the absence of defects, it can only show that software errors are present.

8.1.2. Test Case Design

Any engineering product can be tested in one of two ways:

White Box Testing

This testing is also called as glass box testing. In this testing, by knowing the specified function that a product has been designed to perform test can be conducted that demonstrates each function is fully operation at the same time searching for errors in each function. It is a test case design method that uses the control structure of the procedural design to derive test cases. Basis path testing is a white box testing.

Basis Path Testing

- Flow graph notation
- Cyclomatic Complexity

Deriving test cases Control Structure Testing

- Condition testing

- Data flow testing
- Loop testing

Black Box Testing

In this testing by knowing the internal operation of a product, tests can be conducted to ensure that “all gears mesh”, that is the internal operation performs according to specification and all internal components have been adequately exercised. It fundamentally focuses on the functional requirements of the software.

The steps involved in black box test case design are:

- Graph based testing methods
- Equivalence partitioning
- Boundary value analysis
- Comparison testing
- Graph matrices

8.2. SOFTWARE TESTING STRATEGIES

A Strategy for software testing integrates software test cases into a series of well planned steps that result in the successful construction of software. Software testing is a broader topic for what is referred to as Verification and Validation. Verification refers to the set of activities that ensure that the software correctly implements a specific function. Validation refers to the set of activities that ensure that the software that has been built is traceable to customer's requirements.

8.2.1. Unit Testing

Unit testing focuses verification effort on the smallest unit of software design that is the module. Using procedural design description as a guide, important control paths are tested to uncover errors within the boundaries of the module. The unit test is normally white box testing oriented and the step can be conducted in parallel for multiple modules.

8.2.2. Integration Testing

Integration testing is a systematic technique for constructing the program structure, while conducting test to uncover errors associated with the interface. The objective is to take unit tested methods and build a program structure that has been dictated by design.

- **Top-Down Integration**

Top down integrations is an incremental approach for construction of program structure. Modules are integrated by moving downward through the control hierarchy, beginning with the main control program. Modules subordinate to the main program are incorporated in the structure either in the breath-first or depth-first manner.

- **Bottom-up Integration**

This method as the name suggests, begins construction and testing with atomic modules i.e., modules at the lowest level. Because the modules are integrated in the bottom up manner the processing required for the modules subordinate to a given level is always available and the need for stubs is eliminated.

- **Regression Testing**

In this contest of an integration test strategy, regression testing is the re execution of some subset of test that have already been conducted to ensure that changes have not propagate unintended side effects.

8.2.3. Validation Testing

At the end of integration testing software is completely assembled as a package. Validation testing is the next stage, which can be defined as successful when the software functions in the manner reasonably expected by the customer. Reasonable expectations are those defined in the software requirements specifications. Information contained in those sections form a basis for validation testing approach.

Reasonable expectation is defined in the software requirement specification – a document that describes all user-visible attributes of the software. The specification

contains a section titled “Validation Criteria”. Information contained in that section forms the basis for a validation testing approach.

Validation Test Criteria

Software validation is achieved through a series of black-box tests that demonstrate conformity with requirement. A test plan outlines the classes of tests to be conducted, and a test procedure defines specific test cases that will be used in an attempt to uncover errors in conformity with requirements. Both the plan and procedure are designed to ensure that all functional requirements are satisfied, all performance requirements are achieved, documentation is correct and human-engineered; and other requirements are met.

After each validation test case has been conducted, one of two possible conditions exists: (1) The function or performance characteristics conform to specification and are accepted, or (2) a deviation from specification is uncovered and a deficiency list is created. Deviation or error discovered at this stage in a project can rarely be corrected prior to scheduled completion. It is often necessary to negotiate with the customer to establish a method for resolving deficiencies.

Configuration Review

An important element of the validation process is a configuration review. The intent of the review is to ensure that all elements of the software configuration have been properly developed, are catalogued, and have the necessary detail to support the maintenance phase of the software life cycle. The configuration review sometimes called an audit.

Alpha and Beta Testing

It is virtually impossible for a software developer to foresee how the customer will really use a program. Instructions for use may be misinterpreted. Strange combination of data may be regularly used; and output that seemed clear to the tester may be unintelligible to a user in the field.

When custom software is built for one customer, a series of acceptance tests are conducted to enable the customer to validate all requirements. Conducted by the end user rather than the system developer, an acceptance test can range from an

informal “test drive” to a planned and systematically executed series of tests. In fact, acceptance testing can be conducted over a period of weeks or months, thereby uncovering cumulative errors that might degrade the system over time.

If software is developed as a product to be used by many customers, it is impractical to perform formal acceptance tests with each one. Most software product builders use a process called alpha and beta testing to uncover errors that only the end user seems able to find.

The beta test is conducted at one or more customer sites by the end user of the software. Unlike alpha testing, the developer is generally not present. Therefore, the beta test is a “live” application of the software in an environment that cannot be controlled by the developer. The customer records all problems that are encountered during beta testing and reports these to the developer at regular intervals. As a result of problems reported during beta test, the software developer makes modification and then prepares for release of the software product to the entire customer base.

8.2.4. System Testing

System testing is actually a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated to perform allocated functions.

8.2.5. Security Testing

Attempts to verify the protection mechanisms built into the system.

8.2.6. Performance Testing

This method is designed to test runtime performance of software within the context of an integrated system.

8.3. TEST CASES

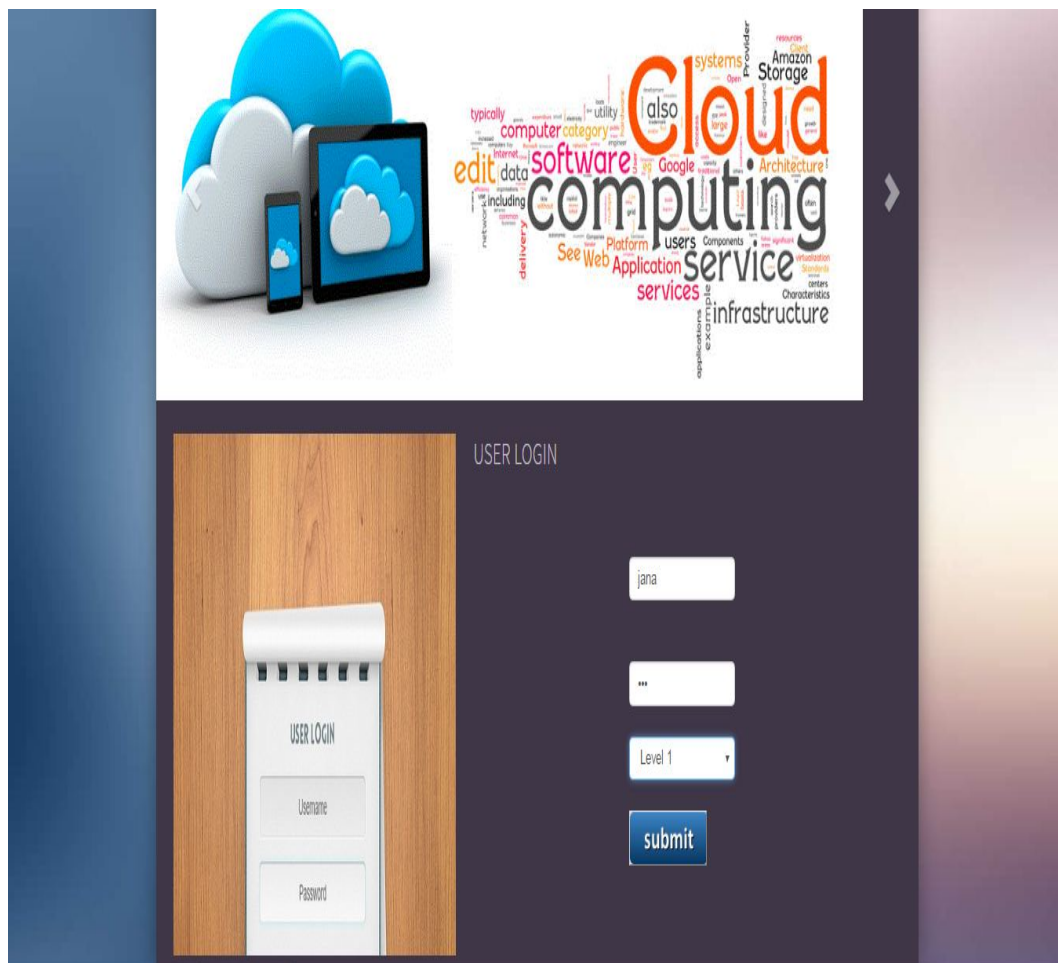
Table 8.1. Test Case Results

Test S.No.	Input	Expected Behavior	Observed behavior	Status P = Passed F = Failed
1	Login as user or admin with correct login details	Administrator or user Home page for manager should be displayed	-do-	P
2	Login as user or admin with wrong login details	Error message should be displayed	-do-	P

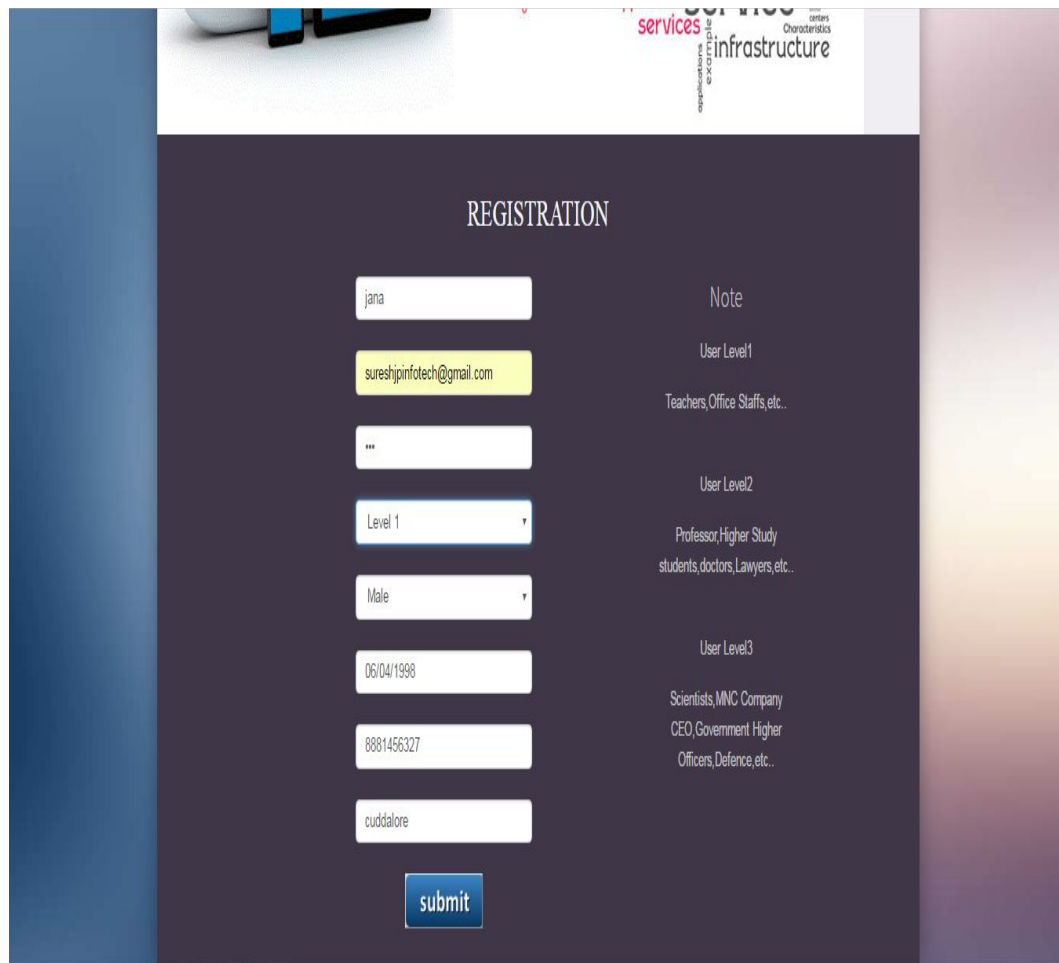
9. SCREENS/ FORMS



Screen1: Home Page



Screen 2: Login Page



The image shows a registration form titled "REGISTRATION" on a dark purple background. The form contains several input fields and a submit button. The fields are: a text field with "jana", a text field with "sureshipinfotech@gmail.com", a text field with "...", a dropdown menu with "Level 1", a dropdown menu with "Male", a text field with "06/04/1998", a text field with "8881456327", and a text field with "cuddalore". Below these fields is a blue "submit" button. To the right of the form, there is a "Note" section with three user levels and their corresponding roles: "User Level1" (Teachers, Office Staffs, etc.), "User Level2" (Professor, Higher Study students, doctors, Lawyers, etc.), and "User Level3" (Scientists, MNC Company CEO, Government Higher Officers, Defence, etc.).

REGISTRATION

jana

sureshipinfotech@gmail.com

...

Level 1

Male

06/04/1998

8881456327

cuddalore

submit

Note

User Level1

Teachers, Office Staffs, etc..

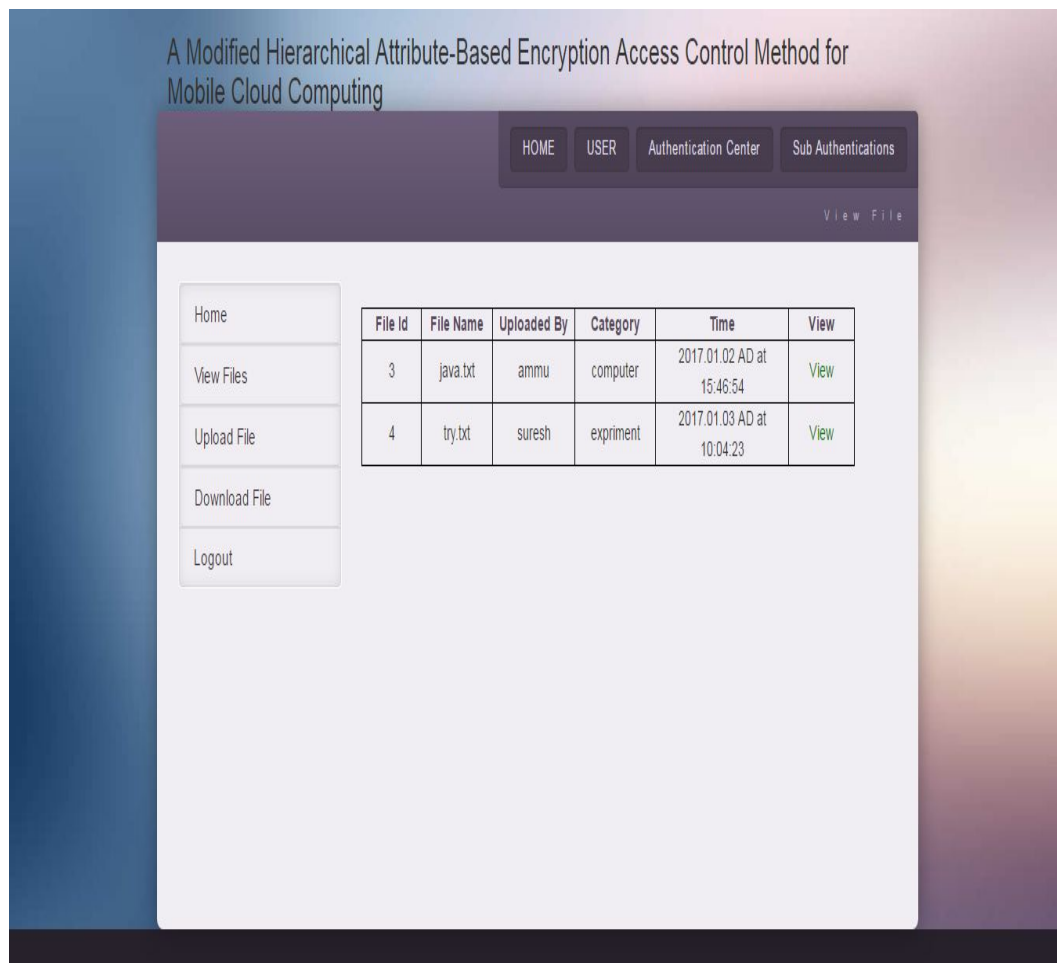
User Level2

Professor, Higher Study students, doctors, Lawyers, etc..

User Level3

Scientists, MNC Company CEO, Government Higher Officers, Defence, etc..

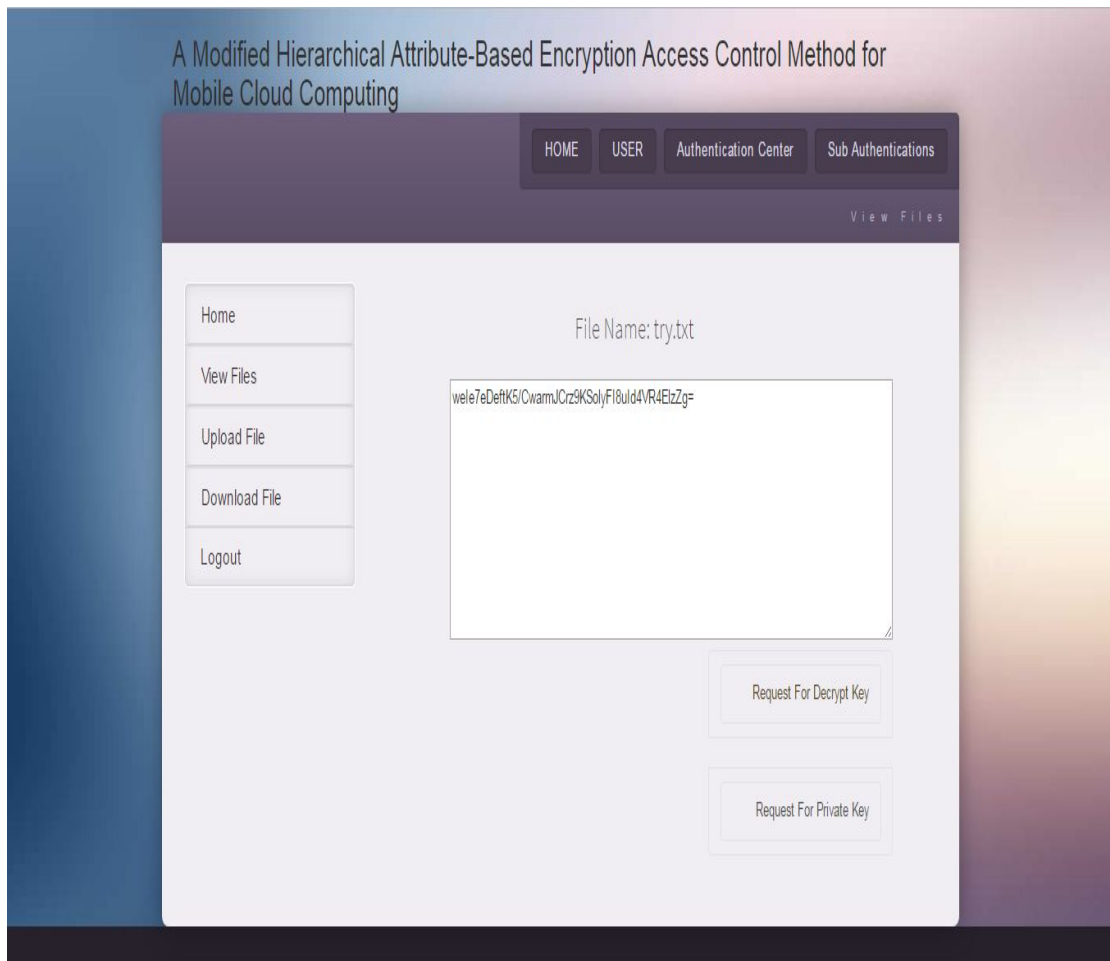
Screen 3: Registration Page



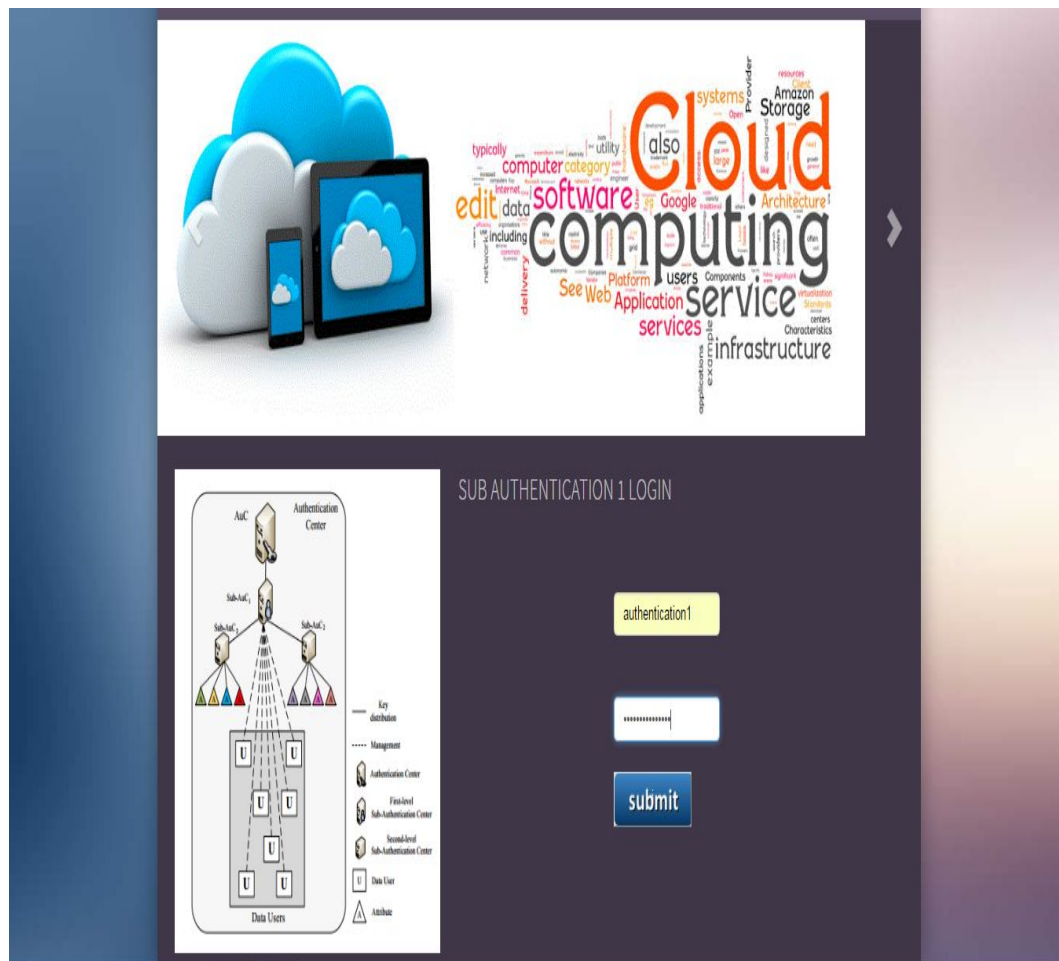
Screen 4: User Home Page

The screenshot displays a web application interface for file uploads. At the top, a dark blue header contains the title "A Modified Hierarchical Attribute-Based Encryption Access Control Method for Mobile Cloud Computing" and a navigation menu with buttons for "HOME", "USER", "Authentication Center", and "Sub Authentications". Below the navigation menu, a breadcrumb trail reads "Data User File Upload". The main content area is titled "File Upload" and features a sidebar on the left with buttons for "Home", "View Files", "Upload File", "Download File", and "Logout". The central form includes a "Choose File" button, a text input field containing "java.txt", a text input field containing "java", a text input field containing "computer language", a dropdown menu currently showing "Level 1", and an "Upload" button.

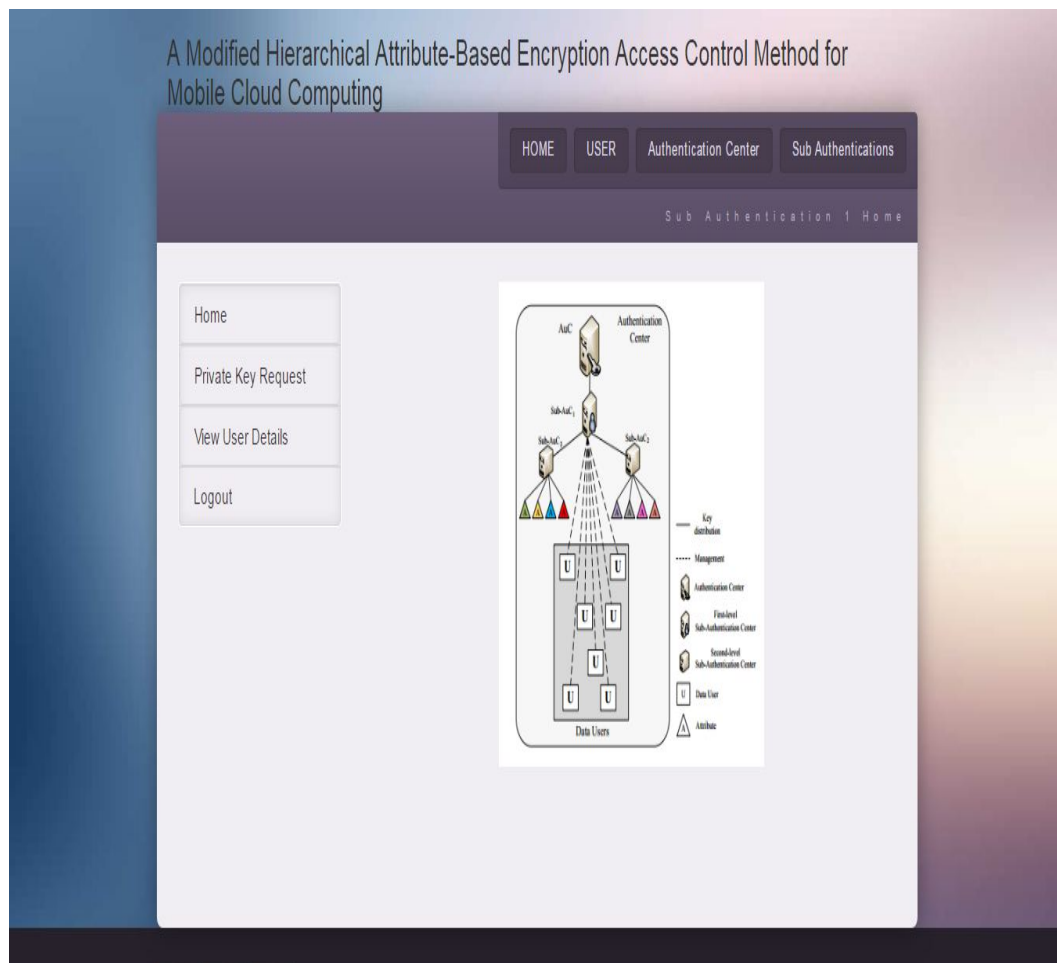
Screen 5: File Upload Page



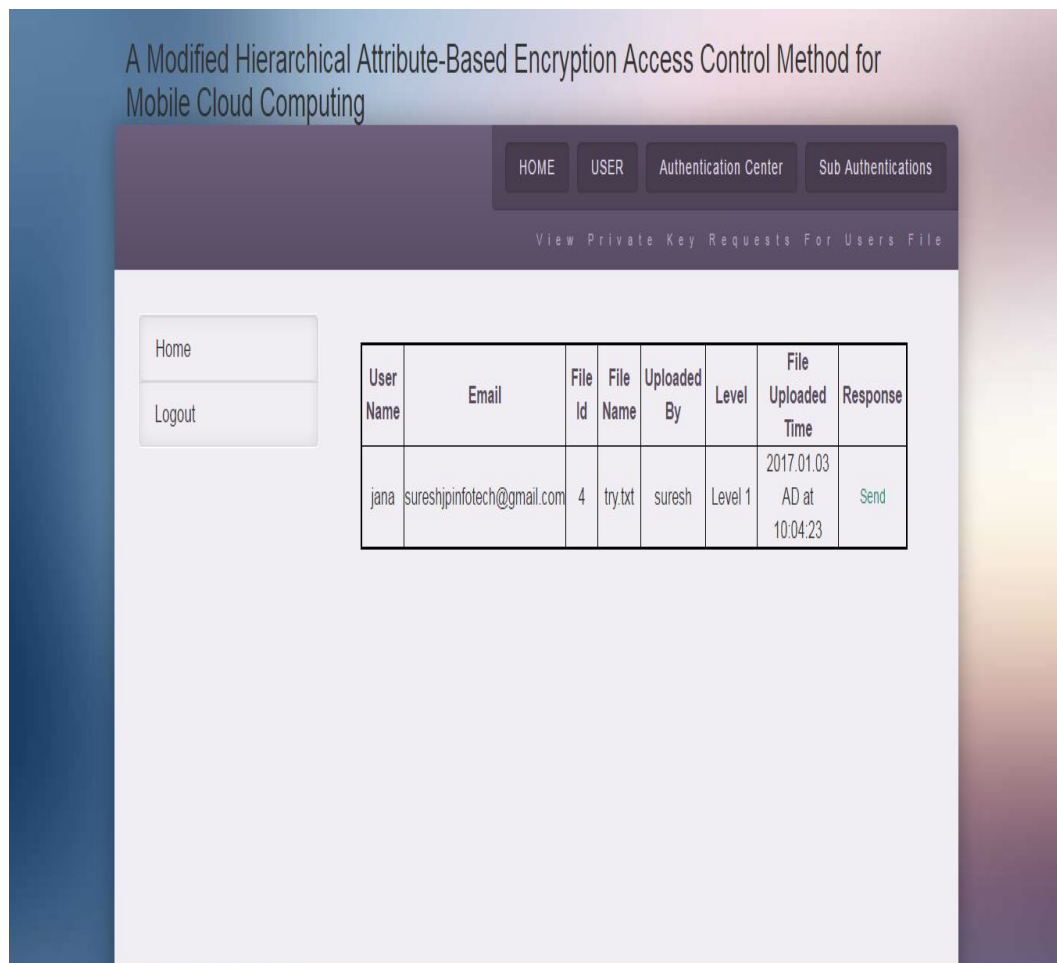
Screen 6: File Encryption



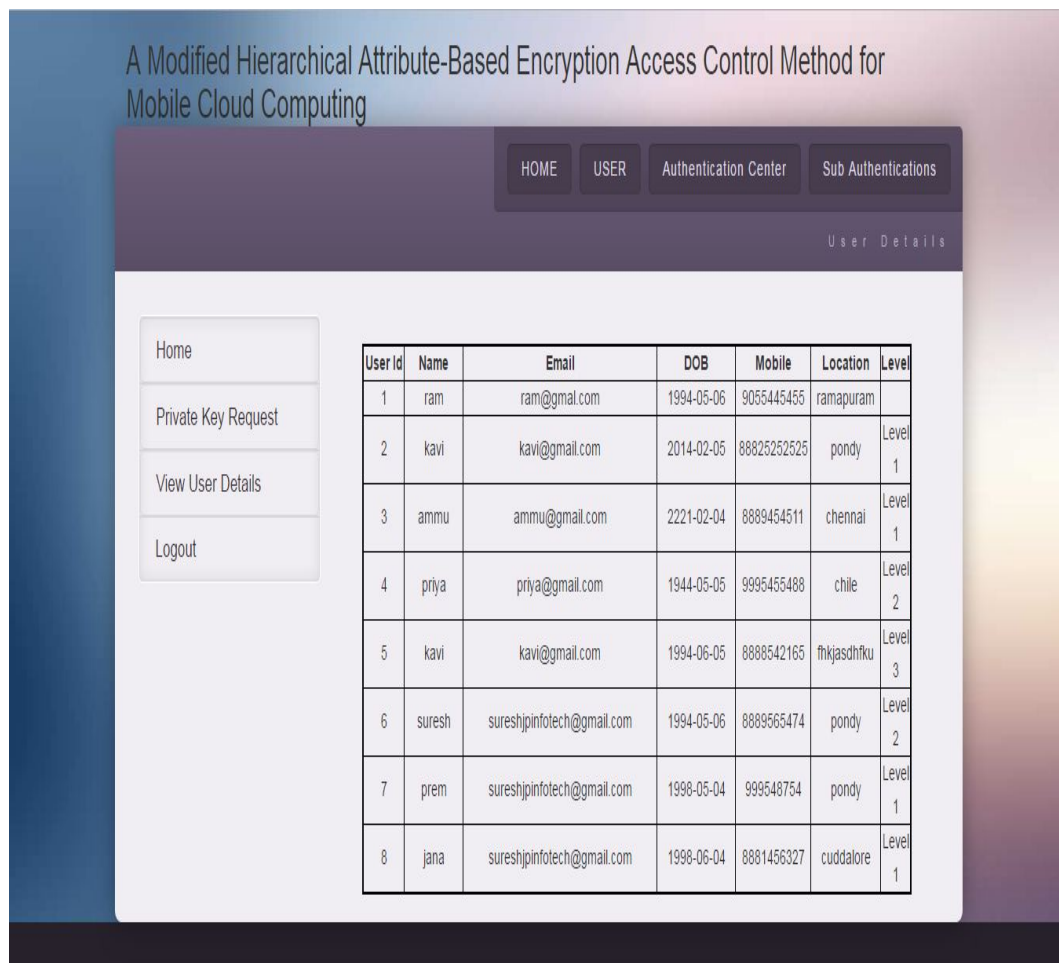
Screen 7: Sub Authentication Login



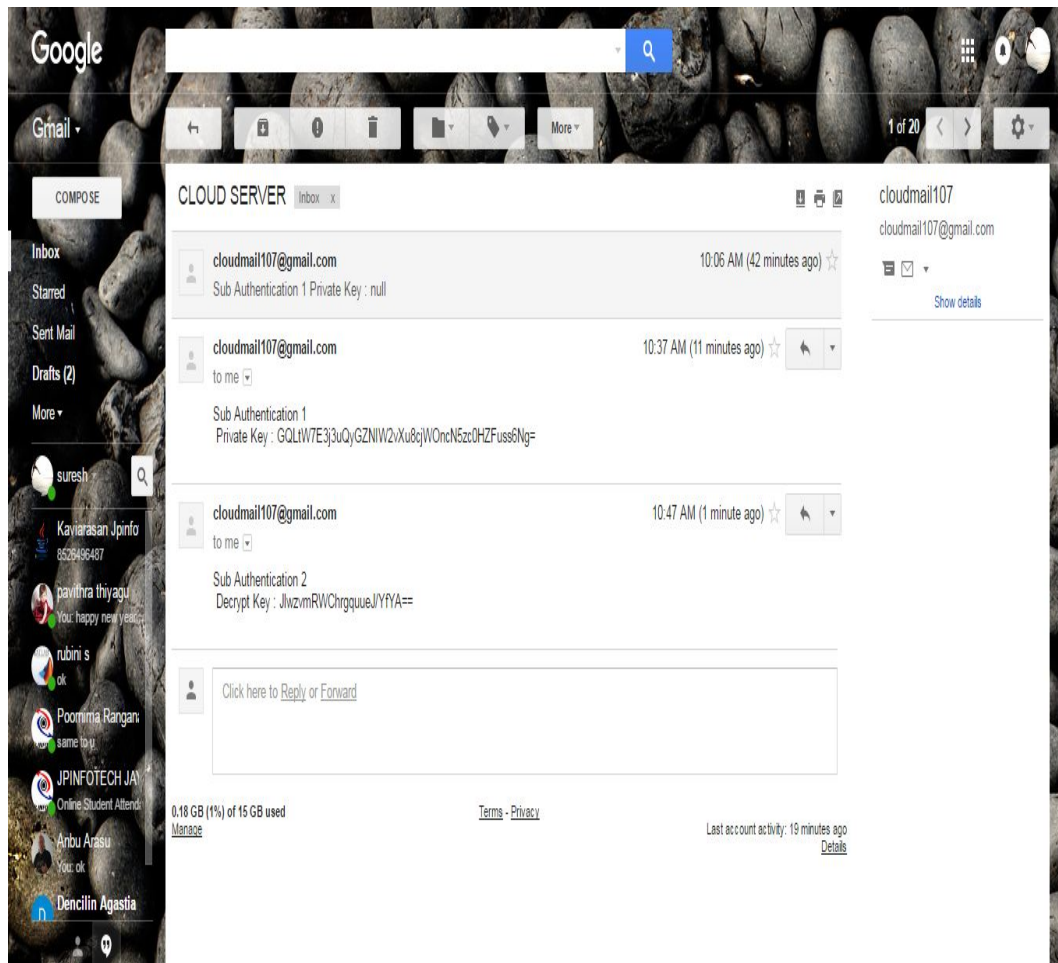
Screen 8: Sub Authentication Menu



Screen 9: User Request Page



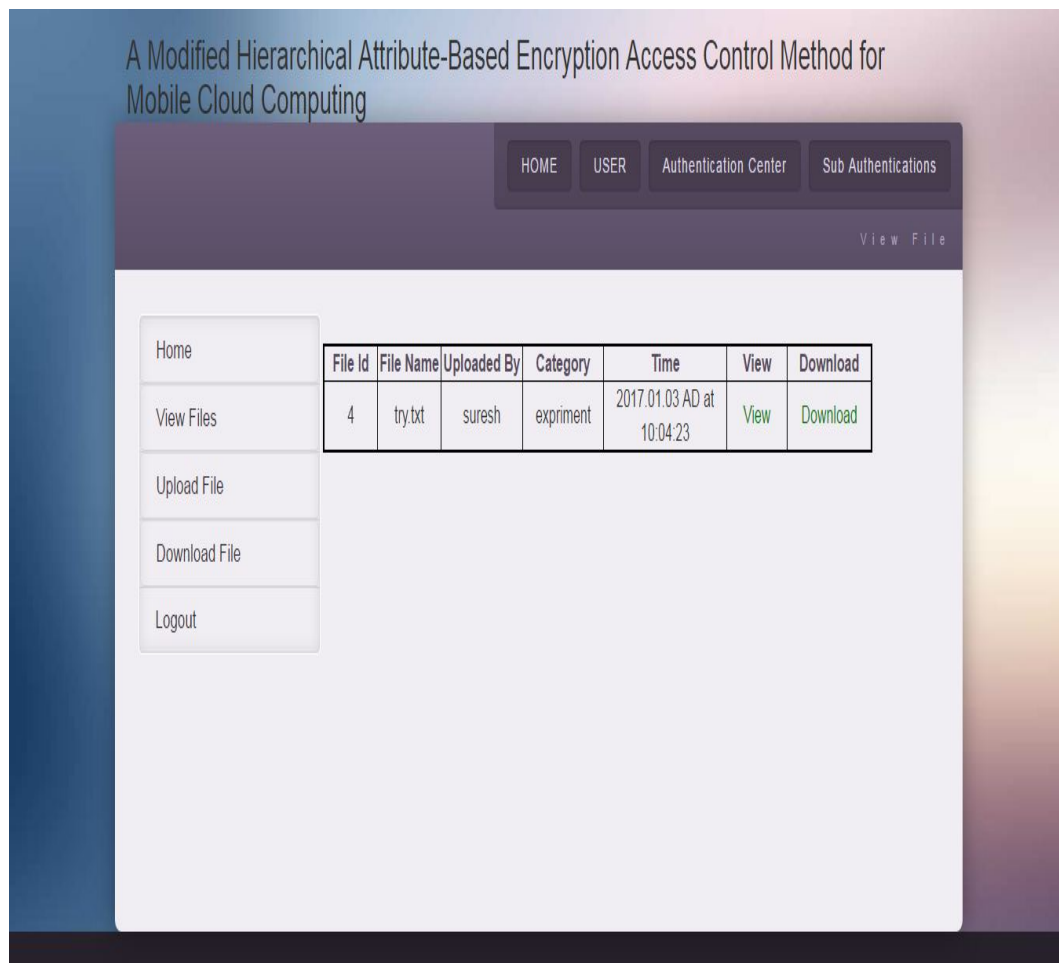
Screen 10: List of Users



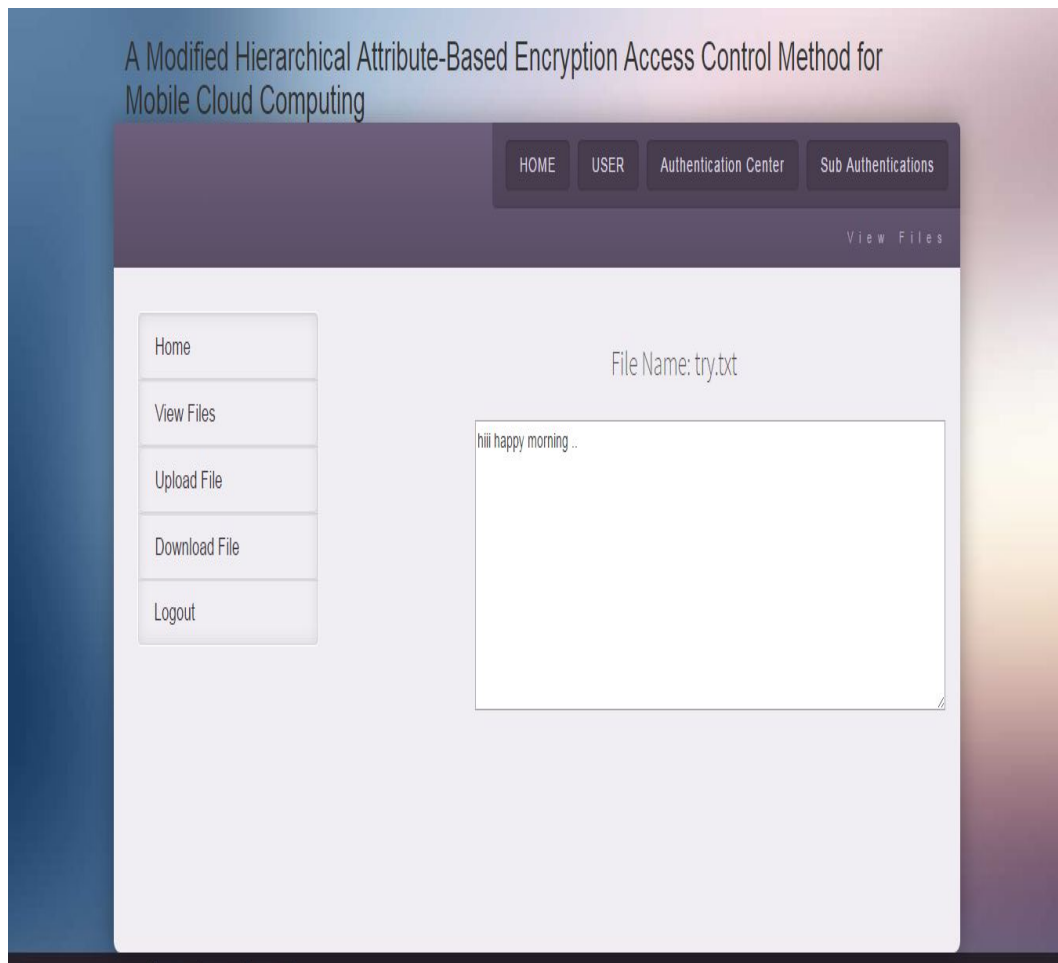
Screen 11:Key to Email

The screenshot displays a web application interface for key verification. At the top, a dark header bar contains the title "A Modified Hierarchical Attribute-Based Encryption Access Control Method for Mobile Cloud Computing" and a navigation menu with buttons for "HOME", "USER", "Authentication Center", and "Sub Authentications". Below the navigation bar, a sub-header reads "Verification For Correct File". The main content area features a left sidebar with buttons for "Home", "View Files", "Upload File", "Download File", and "Logout". The central workspace contains two text input fields: the first is labeled "GQLtW7E3j3uQyGZNIW2vXu8cjWk" and the second is labeled "JlwzvmRWChrgguuJYfYA==". A "Verify" button is positioned below the second input field.

Screen 12: Key Verification



Screen 13: Download Page



Screen 14: Download Text

10. FUTURE ENHANCEMENT

Future research avenues of this work include investigation The scheme not only accomplishes the hierarchical access control of mobile sensing data in the mobile cloud computing model, but protects the data from being obtained by an untrusted third party

11. CONCLUSION

The Project proposed a modified HABE scheme by taking advantages of attributes based encryption (ABE) and hierarchical identity based encryption (HIBE) access control processing. The proposed access control method using MHABE is designed to be utilized within a hierarchical multiuser data-shared environment, which is extremely suitable for a mobile cloud computing model to protect the data privacy and defend unauthorized access. Compared with the original HABE scheme, the novel scheme can be more adaptive for mobile cloud computing environment to process, store and access the enormous data and files while the novel system can let different privilege entities access their permitted data and files. The scheme not only accomplishes the hierarchical access control of mobile sensing data in the mobile cloud computing model, but protects the data from being obtained by an untrusted third party

12. BIBLIOGRAPHY

- [1] D. Bauer, S. Rooney, and P. Scotton, "Network infrastructure for massively distributed games," in Proc. ACM 1st Workshop Netw. Syst. Support Games, 2002, pp. 36–43.
- [2] S. D. Webb, S. Soh, and W. Lau, "Enhanced mirrored servers for network games," in Proc. ACM 6th ACM SIGCOMM Workshop Netw. Syst. Support Games, 2007, pp. 117–122.
- [3] L. Ahmad, A. Boukerche, A. A. Hamidi, A. Shadid, and R. Pazzi, "Web-Based e-learning in 3d large scale distributed interactive simulations using HLA/RTI," in Proc. IEEE Int. Symp. Parallel Distrib. Process., 2008, pp. 1–4.
- [4] Agustina, F. Liu, S. Xia, H. Shen, and C. Sun, "CoMaya: Incorporating advanced collaboration capabilities into 3D digital media design tools," in Proc. ACM Conf. Comput. Supported Cooperative Work, 2008, pp. 5–8.
- [5] F. Safaei, P. Boustead, C. D. Nguyen, J. Brun, and M. Dowlatshahi, "Latency-driven distribution: Infrastructure needs of participatory entertainment applications," IEEE Commun. Mag., vol. 43, no. 5, pp. 106–112, May 2005.
- [6] M. Mauve, J. Vogel, V. Hilt, and W. Effelsberg, "Local-lag and timewarp: Providing consistency for replicated continuous applications," IEEE Trans. Multimedia, vol. 6, no. 1, pp. 47–57, Feb. 2004.
- [7] V. Valancius, N. Laoutaris, L. Massoulie, C. Diot, and P. Rodriguez, "Greening the internet with nano data centers," in Proc. ACM 5th Int. Conf. Emerging Netw. Exp. Technol., 2009, pp. 37–48.
- [8] S. Choy, B. Wong, G. Simon, and C. Rosenberg, "The brewing storm in cloud gaming: A measurement study on cloud to enduser latency," in Proc. ACM 11th Annu. Workshop Netw. Syst. Support Games, 2012, pp. 1–6.
- [9] D. Delaney, T. Ward, and S. McLoone, "On consistency and network latency in distributed interactive applications: A survey part i," Presence: Teleoperators Virtual Envir., vol. 15, no. 2, pp. 218–234, 2006.